B.Sc. (Hons) in Software Development

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Simplified Cross-Platform HRM for Puzzle-Solving

By
Kyrylo Kozlovskyi

April 26, 2026

Minor Dissertation

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Context

Reasoning, understood as the capacity to plan and execute complex sequences of actions under constraints, remains one of the hardest open problems in artificial intelligence. The dominant approach to reasoning in modern AI relies on Large Language Models (LLMs) such as GPT-5.2, Claude Opus 4.6, and DeepSeek R1, which employ Chain-of-Thought (CoT) prompting to generate intermediate textual steps before producing a final answer [3]. Although CoT has improved benchmark performance in many domains, it introduces fundamental weaknesses. Errors made early in the chain cascade into subsequent steps [4, 5], and problems that require backtracking or parallel exploration quickly become intractable. More critically, the fixed-depth Transformer architecture constrains these models to complexity classes that cannot efficiently solve even polynomial-time problems [6]. This is not a theoretical curiosity: leading CoT models as of August 2025 score 0% on expert-level Sudoku puzzles that require systematic backtracking [1, 7].

The Hierarchical Reasoning Model (HRM), introduced by Wang *et al.* [1] at Sapient Inc., takes a fundamentally different approach. Drawing on hierarchical processing in the mammalian brain, HRM operates in continuous latent space through two coupled recurrent modules: a high-level (H) module for slow, abstract planning and a low-level (L) module for rapid, detailed computation. Instead of generating tokens one at a time, the model iteratively refines a latent representation of the solution until it converges. With only 27 million parameters and 1,000 training examples, HRM achieves 40.3% accuracy on ARC-AGI-1, outperforming models with hundreds of billions of parameters, and solves 55% of expert-difficulty Sudoku puzzles where all tested CoT models fail entirely [1].

However, subsequent work by Ge *et al.* [8], corroborated by the ARC Foundation [9], suggests that HRM's success may derive primarily from computational depth and iterative refinement rather than from the hierarchical structure itself. An L-module-only variant achieves comparable accuracy while training significantly faster, motivating investigation of whether the simpler architecture suffices. These findings are discussed in detail in Chapter 3.

Despite its compact size, the official Sapient Inc. implementation is tightly coupled to CUDA and NVIDIA GPUs, leaving no platform-agnostic implementation available for educational or experimental use, and no empirical data on deployment to resource-constrained edge devices. Edge deployment of compact reasoning models matters for several practical reasons: privacy-sensitive applications benefit from on-device inference without transmitting data to cloud servers, educational settings often lack access to GPU

hardware, and IoT or embedded scenarios require offline reasoning capability within strict power and latency budgets.

1.2 Project Overview

This project presents an independent, cross-platform implementation of the Hierarchical Reasoning Model, developed as a pair project by Fionn McCarthy and Kyrylo Kozlovskyi. The implementation is based on the architecture described by Wang *et al.* [1] and the official Sapient Inc. codebase[1], with several extensions. A Simplified HRM (L-Module Only) variant was implemented from scratch, deliberately designed for cross-platform execution across GPU, CPU, and ARM platforms. The architectural details and design rationale are described in Chapter 4.

Alongside the reimplementation, the project introduces a Weighted Maze task in which each traversable cell in a 15×15 grid carries an associated movement cost, requiring the model to find a path that minimises total cost rather than step count. Ground-truth solutions are computed via Dijkstra's algorithm. This task has not previously been evaluated with any HRM variant. The project additionally investigates deployment of the trained model on a Raspberry Pi 5 to assess the viability of HRM inference on edge hardware.

My individual contributions to this project focus on:

1. Implementing the L-module-only (Simplified HRM) variant from scratch, including the fixed-point iteration loop with self-conditioning feedback, eight weight-shared transformer blocks with SwiGLU activation, and rotary positional embeddings.

2. Building the continuous integration and code quality pipeline using GitHub Actions, enforcing Black formatting, Ruff linting, and pytest test execution on every commit.

3. Designing and implementing the ONNX export pipeline (`export_onnx.py`), including a `SimplifiedHRMForONNX` wrapper class that resolves six ONNX-incompatible patterns in the model to enable conversion at opset 18.

4. Developing a zero-dependency inference script (`solve_onnx.py`) that runs trained models using only NumPy and ONNX Runtime, with no PyTorch installation required.

5. Deploying and benchmarking the ONNX model on a Raspberry Pi 5 running Debian 13 (Trixie), providing the first empirical data on HRM inference performance on ARM-based edge hardware.

Fionn McCarthy led the implementation of the full HRM model (both H-module and L-module), the Weighted Maze generator and Dijkstra solver, the Sudoku dataset generator, the training pipeline with deep supervision and one-step gradient approximation, the metrics and logging infrastructure, the trajectory visualiser, and the multi-seed evaluation analysis.

The project has been developed since October 2025 and is publicly available at `https://github.com/fionntmcc/cross-platform-hrm`.

[1] `https://github.com/sapientinc/HRM/`

1.3 Objectives

The project pursues the following objectives:

1. **Implement a functional Hierarchical Reasoning Model.** Build a working HRM with both the H-module and L-module, capable of training from scratch and performing inference on grid-based reasoning tasks, using a pure PyTorch codebase that avoids CUDA-specific extensions.

2. **Implement an L-module-only variant.** Construct a variant of the HRM that removes the high-level module entirely, preserving only the low-level recurrent module with eight weight-shared transformer blocks and 16 iterative reasoning steps, in order to investigate whether hierarchical structure is necessary for effective latent reasoning.

3. **Train and benchmark on Sudoku puzzles.** Train the L-module-only variant on 9×9 Sudoku puzzles of mixed difficulty (easy through expert-level, requiring zero to 50+ backtracks) and evaluate puzzle accuracy, token accuracy, and convergence behaviour.

4. **Train and benchmark on a novel Weighted Maze task.** Evaluate whether the HRM architecture can generalise to cost-sensitive pathfinding on 15×15 grids with heterogeneous traversal costs, a task not previously explored in the literature. A target accuracy of 60% on 15×15 mazes was agreed upon as a sufficient benchmark for success.

5. **ONNX export and edge deployment on Raspberry Pi 5.** Export the trained model to ONNX format, resolving the technical challenges of representing an iterative reasoning loop in a static computation graph. Deploy the ONNX model on a Raspberry Pi 5 running Debian 13 (Trixie) and benchmark inference latency using ONNX Runtime with zero PyTorch dependency.

6. **Continuous integration and code quality.** Implement a CI/CD pipeline using GitHub Actions to enforce code quality through automated formatting (Black), linting (Ruff), and testing (pytest) on every commit, ensuring consistent quality across both contributors.

1.4 Dissertation Structure

The remainder of this dissertation is organised as follows. Chapter 2 describes the research and software development methodologies used throughout the project, including the Agile workflow, code review process, CI/CD pipeline, and division of work. Chapter 3 presents a technology review covering HRM architecture, Chain-of-Thought reasoning limitations, critical perspectives on hierarchical structure, cross-platform inference frameworks, ONNX conversion challenges, and edge deployment considerations. Chapter 4 details the system design, including both model variants, the task implementations, the ONNX export pipeline and its six conversion blockers, and the training infrastructure. Chapter 5 evaluates the system against the objectives listed above, presenting benchmark results across GPU, CPU, and Raspberry Pi hardware alongside a discussion of strengths

and limitations. Chapter 6 summarises the findings and identifies directions for future work.

The full source code, documentation, and a screencast demonstration are available at `https://github.com/fionntmcc/cross-platform-hrm`. Appendix A provides a summary of the repository contents and installation instructions.

Chapter 2

Methodology

This chapter describes the research methodology, software development methodology, and experimental methodology employed throughout the project. It also outlines the tools, hardware, and division of work between both team members.

2.1 Research Methodology

The research component of this project follows a structured literature review approach guided by three questions: (1) What architectural mechanisms enable HRM's performance on definitive reasoning tasks? (2) What are the practical challenges of deploying iterative reasoning models on resource-constrained edge hardware? (3) What limitations and open questions remain regarding HRM's design and cross-platform applicability?

Sources were identified through academic databases including IEEE Xplore, Scopus, arXiv, and ACM Digital Library, supplemented with technical reports from AI research organisations and hardware vendor documentation. Given the recent emergence of the HRM, the review includes preprints alongside peer-reviewed publications. Inclusion criteria required sources to address recurrent or novel reasoning architectures, provide empirical or theoretical analysis of cross-platform inference frameworks, or evaluate neural network deployment on ARM-based edge devices. Sources published between 2021 and 2025 were prioritised, with earlier works included where they were foundational.

Both team members conducted independent literature reviews prior to the implementation phase. My review examined cross-platform inference frameworks with emphasis on ONNX Runtime, edge deployment challenges on ARM hardware, quantisation techniques for iterative architectures, and Raspberry Pi 5 hardware capabilities, covering 22 sources with empirical results. Fionn McCarthy's review focused on HRM architecture, performance benchmarks, comparative analysis with CoT LLMs, critical perspectives from Ge *et al.* [8], biological correspondence, and energy implications, covering 23 sources. These two reviews are synthesised in Chapter 3.

A primary limitation of the research methodology is the premature state of HRM literature. Only two substantial technical papers directly address the architecture [1, 8], with limited independent replication. This constraint necessitated drawing on related work from adjacent fields, including Deep Equilibrium Models [10], Adaptive Computation Time [11], and recurrent depth approaches [12], to contextualise findings.

2.2 Software Development Methodology

The project followed an Agile development methodology with iterative sprints, beginning in October 2025. Work was organised using a Kanban board within GitHub Projects, providing visibility over task progress across both team members. Each unit of work was captured as a GitHub Issue, and development followed a branch-per-issue workflow: for every issue, a dedicated branch was created, worked on, and merged back into the main branch via a Pull Request (PR) upon completion. Weekly supervisor meetings with Dr. John Healy provided oversight and feedback throughout the academic year. Figure 2.1 shows the resulting commit history across both contributors.

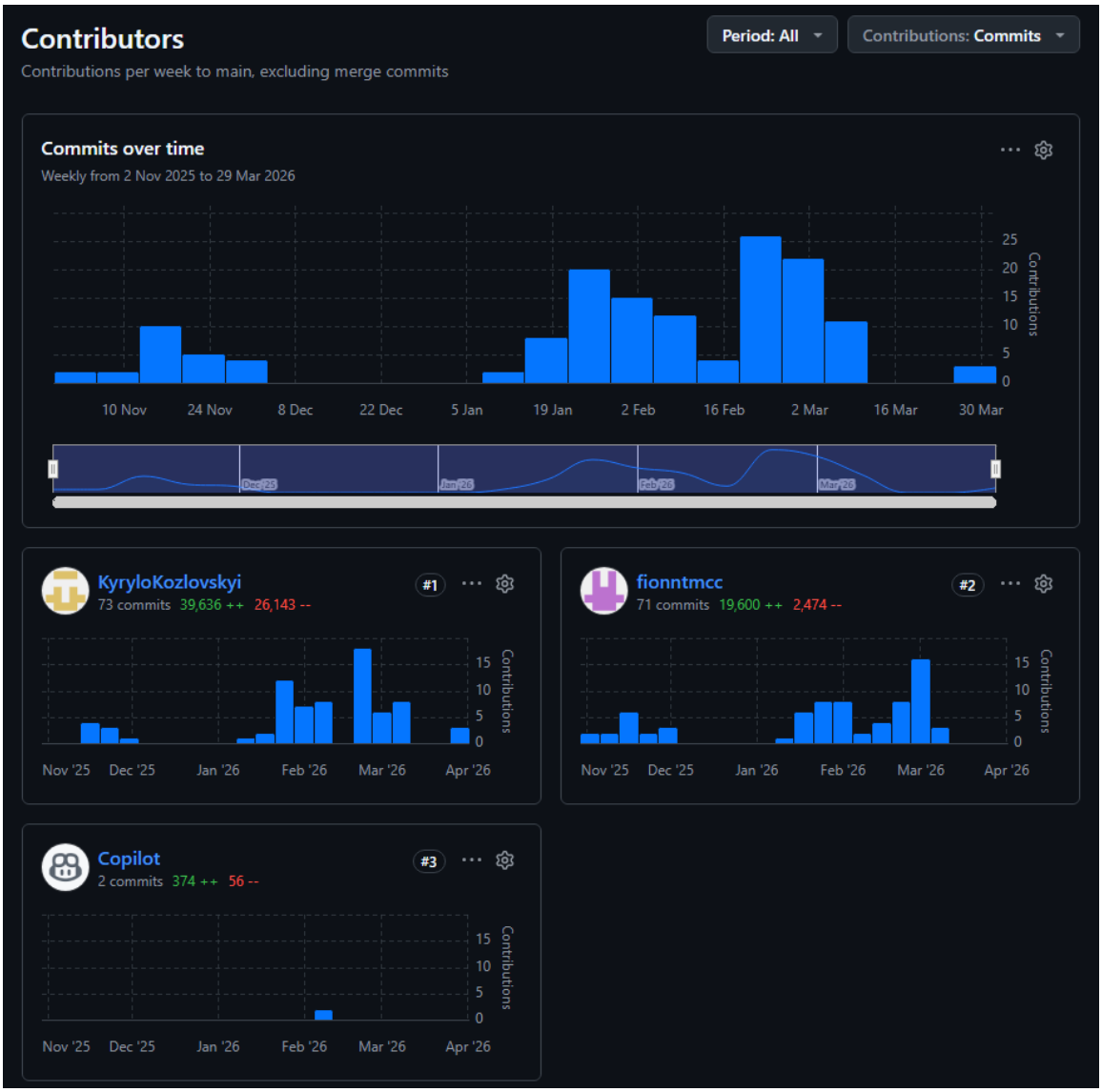


Figure 2.1: Repository commit history showing the distribution of contributions between both team members across November 2025 to March 2026.

2.2.1 Code Review and Quality Assurance

A code review process was followed throughout the project. Every Pull Request required review before merging. I reviewed Fionn McCarthy's PRs and he reviewed mine, ensuring

that both team members maintained familiarity with the full codebase and that no code entered the main branch without scrutiny. In addition to human review, GitHub Copilot was configured to automatically review every Pull Request, providing an additional layer of automated feedback on code quality, potential bugs, and style consistency.

I implemented a CI pipeline on GitHub Actions which enforced Black formatting and Ruff linting on every push, as well as running the pytest test suite. This ensured a consistent code structure and quality across the codebase. Pre-commit hooks were also configured to catch formatting issues before they reached the remote repository.

2.2.2 Iterative Architecture Correction

An important methodological note concerns the evolution of the model architecture during the early stages of development. An initial prototype built in November 2025 adopted HRM-inspired naming and module structure but did not faithfully reproduce the iterative reasoning loop described by Wang *et al.* [1]. The prototype stacked multiple Worker modules sequentially rather than applying a single weight-shared module iteratively, which collapsed the intended fixed-point iteration into a deeper feedforward computation.

The error was identified through close re-reading of the Ge *et al.* [8] ablation paper, which clarified that the "8-layer L-module" described in the original work refers to the internal transformer depth of a single weight-shared module applied iteratively, rather than eight separately stacked modules. Correcting this distinction reshaped subsequent development of both the Full HRM and the L-module-only variant. This is documented here as a genuine methodological learning outcome about the importance of precise terminology when reproducing recent research architectures.

2.2.3 Work Packages and Division of Labour

Work was divided into discrete work packages assigned to each team member based on their areas of focus. Table 2.1 summarises the primary responsibilities.

While primary responsibilities were assigned as shown, both team members contributed across all areas through the code review process and collaborative debugging.

2.2.4 Version Control and Repository Structure

The project is hosted on GitHub at `https://github.com/fionntmcc/cross-platform-hrm`. The repository is structured as a Python package under the `hrm/` directory, with separate directories for tests (`tests/`), documentation (`docs/`), scripts (`scripts/`), and CI/CD workflows (`.github/workflows/`). A GitHub Actions pipeline was configured to run automated tests on each push, ensuring that changes did not introduce regressions.

The repository contains 131 commits at the time of writing (62 from me, 65 from Fionn McCarthy, and 4 automated commits from Copilot), spanning from the initial commit in November 2025 through to the final ONNX inference work in March 2026. Development activity clustered into four phases: project setup and 4×4 prototyping in November 2025, core HRM module implementation through January 2026, the L-module-only variant alongside training infrastructure and CI/CD in February 2026, and Weighted Maze integration with ONNX export and metrics logging in March 2026. Figure 2.2 provides an overview of the project timeline.

Work Package	Fionn McCarthy	Kyrylo Kozlovskyi
WP1: Literature Review	HRM architecture, benchmarks, CoT comparison, critical perspectives, biological correspondence, energy implications	Edge deployment, ONNX, ARM hardware, quantisation, Raspberry Pi 5
WP2: Core Architecture	Lead developer of Full HRM, Worker module with gating, halting mechanism (ACT), outer iteration loop, transformer components, UnifiedHRM assembly	L-module-only (Simplified HRM) variant, project setup, RMSNorm, InputNetwork, PlannerModule, OutputNetwork, fixed-point iteration loop
WP3: Dataset Generation	4×4 and 9×9 Sudoku generators with difficulty levels, heuristic maze generator, Weighted Maze generator with Dijkstra solver, encoding and formatting standardisation	SudokuDataset PyTorch class, validation utilities, dataset export/import CLI (JSON/CSV)
WP4: Training & Evaluation	Training pipeline design, metrics and logger modules, trajectory visualiser, multi-seed analysis, benchmark collection	Training run execution, maze visualiser, demo and run scripts
WP5: Deployment	Vocab parameterisation for multi-task support	ONNX export pipeline, zero-dependency inference script, Raspberry Pi 5 deployment and benchmarking
Infrastructure	README	GitHub Actions CI (Black, Ruff, pytest), pre-commit hooks
WP6: Dissertation	Introduction, Methodology, Technology Review, Design	Evaluation, Conclusion, formatting

Table 2.1: Division of work packages between team members.

2.3 Experimental Methodology

The experimental methodology was designed to enable a controlled comparison between the full HRM (with both H-module and L-module) and the L-module-only variant across two reasoning tasks: Sudoku and Weighted Maze.

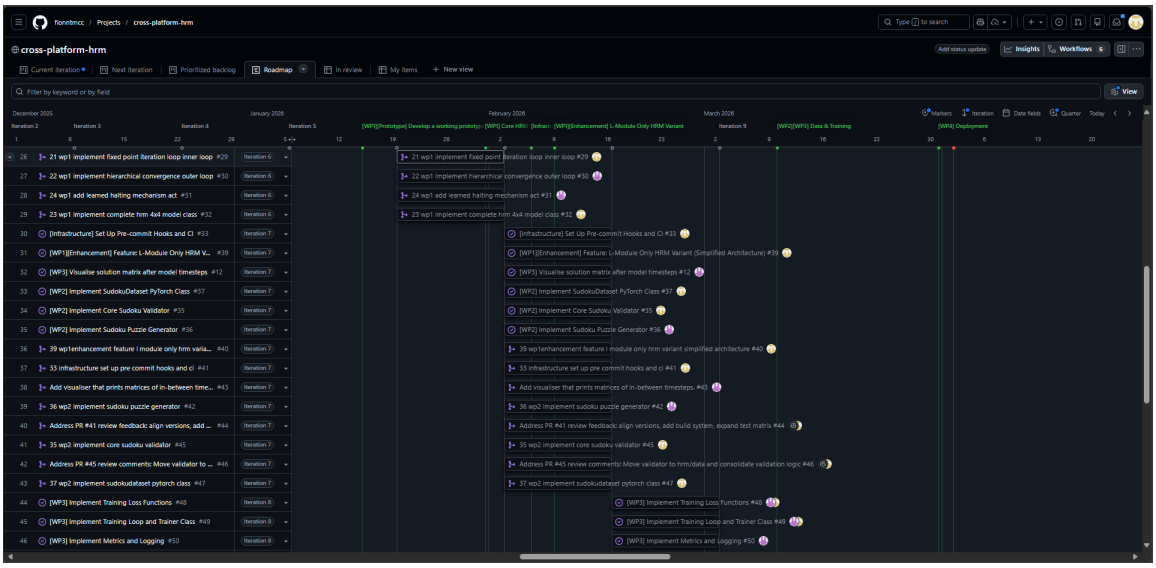


Figure 2.2: Project roadmap showing the timeline of work packages from October 2025 to April 2026.

2.3.1 Independent and Dependent Variables

The primary independent variables across all experiments are the model architecture (full HRM versus L-module-only), the task (Sudoku versus Weighted Maze), and the inference platform (GPU, CPU, Raspberry Pi 5). For each task, both variants were trained using identical hyperparameters, datasets, and training procedures, isolating the effect of the hierarchical structure on performance.

The dependent variables measured are:

- **Puzzle Accuracy**: the percentage of test puzzles solved with every cell correct.
- **Token/Cell Accuracy**: the percentage of individual cells predicted correctly.
- **Training time**: wall-clock time to reach convergence or a fixed number of epochs.
- **Convergence behaviour**: training and validation loss curves over epochs.
- **Inference latency**: time per puzzle at inference, measured across GPU, CPU, and Raspberry Pi hardware.
- **Generalisation**: accuracy on unseen data generated with different random seeds, particularly relevant for the Weighted Maze task.

2.3.2 Training Protocol

All models were trained from random initialisation on the same training pipeline using a GIGABYTE A16 PRO DXH laptop (Intel Core 7 240H, 10C/16T @ 5.2 GHz; NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12 GB GDDR7; 32 GB LPDDR5; 1 TB Samsung NVMe; Windows 11). This machine belongs to Fionn McCarthy. The PyTorch framework was used without additional CUDA extensions such as FlashAttention, as the project prioritised cross-platform compatibility over raw training speed. Mixed-precision training (AMP) was enabled throughout.

Four model checkpoints were produced: Sudoku 4×4 (150 epochs), Sudoku 9×9 (1,000 epochs), Maze 11×11 (116 epochs, 87.2% validation accuracy), and Maze 15×15 (187 epochs, 66.8% validation accuracy). All four were exported to ONNX format, yielding eight model files in total (four `.pt` checkpoints and four `.onnx` graphs with accompanying `.onnx.data` external weight files). Specific hyperparameter values are reported alongside results in Chapter 5.

2.3.3 Evaluation Protocol

Evaluation was performed on held-out test sets not seen during training. For Sudoku, the evaluation dataset uses mixed difficulty with seed 99. For Maze, evaluation uses unseen layouts (different random seed from training), also seed 99. Accuracy was measured at both the puzzle level (every cell correctly filled) and the token level (percentage of individual cells correct). For Maze tasks, path-specific metrics were also computed: path precision, path recall, and path F1 score.

ONNX inference evaluation used 500 puzzles per task. PyTorch CPU inference used 200 puzzles per task. Accuracy was confirmed to be identical across all three hardware platforms (GPU, laptop CPU via ONNX Runtime, and Raspberry Pi 5 via ONNX Runtime), verifying that the ONNX export preserves model semantics.

Inference was benchmarked across three hardware configurations to evaluate the model's portability:

- **GPU**: NVIDIA GeForce RTX 5070 Ti Laptop GPU with 12 GB GDDR7 (Fionn McCarthy's GIGABYTE A16 PRO DXH, used for training).
- **CPU**: Intel i7-12700H (14C/20T @ 4.7 GHz) without GPU acceleration, via ONNX Runtime (my ASUS ROG Zephyrus M16 laptop, 16 GB DDR5-4800, 1 TB NVMe).
- **Edge device**: Raspberry Pi 5 Model B Rev 1.1 with 16 GB LPDDR4X, 32 GB microSD, running Debian 13 (Trixie), inference via ONNX Runtime only.

This range of hardware allows the project to assess whether HRM's compact architecture translates into practical deployability beyond conventional GPU setups.

2.4 Technology Choice Justification

Several technology choices were made deliberately and merit brief justification. PyTorch was chosen over TensorFlow because the official HRM reference implementation by Sapient Inc. uses PyTorch, which simplified validation against the original codebase. ONNX was chosen over TensorRT (which is NVIDIA-only and therefore unsuitable for ARM deployment) and CoreML (which is Apple-only) because it is the only format that targets both x86 and ARM platforms through a single export. GitHub Actions was chosen for CI/CD over alternatives such as Jenkins or CircleCI because it integrates directly with the GitHub repository, requires no additional infrastructure, and provides free build minutes for open-source projects.

2.5 Ethical Considerations

This project does not involve human participants, personal data, or sensitive information. All datasets are synthetically generated and contain no personally identifiable information. The project uses publicly available open-source software and builds upon research published under open-access licences. No ethical approval was required.

2.6 Tools and Technologies

Table 2.2 summarises the tools and technologies used throughout the project.

Tool / Technology	**Purpose**
Python 3	Implementation language
PyTorch	Deep learning framework for model training
ONNX / ONNX Runtime	Cross-platform model export (opset 18) and inference
GitHub	Version control, issue tracking, project management
GitHub Actions	Continuous integration and automated testing
GitHub Copilot	Automated Pull Request review
Black / Ruff	Python code formatting and linting
pytest	Unit and integration testing
LaTeX (Overleaf)	Dissertation typesetting
Raspberry Pi 5 (16 GB)	Edge device for ONNX inference testing
NVIDIA RTX 5070 Ti (12 GB)	GPU for model training (Fionn's laptop)
NVIDIA RTX 3060 (6 GB)	Inference laptop GPU (my laptop)

Table 2.2: Tools and technologies used in the project.

Chapter 3

Technology Review

This chapter synthesises the literature on Hierarchical Reasoning Models, drawing on two independent reviews conducted by both team members. My review examined cross-platform inference frameworks, edge deployment challenges, quantisation techniques for iterative architectures, and ARM hardware capabilities across three domains: HRM architectures (7 sources), edge optimisation (8 sources), and cross-platform frameworks (5 sources), totalling 22 sources with empirical results. Fionn McCarthy's review examined HRM architecture, empirical performance, comparative analysis with CoT LLMs, critical perspectives, and biological correspondence. The sections that follow integrate findings from both reviews into a unified technology review organised by theme.

3.1 Chain-of-Thought Reasoning and Its Limitations

Contemporary Large Language Models tackle complex problems through Chain-of-Thought (CoT) prompting [3], where sequential tokens are generated prior to the final response. While this makes their logic transparent and sometimes effective, it suffers from brittleness. One wrong step can cascade through the entire chain [4, 5], and problems requiring parallel exploration or backtracking become computationally intractable. According to the International Energy Agency, LLMs accounted for 415 TWh of energy, or 1.5% of global energy usage as of January 2025 [13].

Furthermore, the fixed-depth Transformer architecture constrains these models to complexity classes that cannot handle polynomial-time problems efficiently [6]. State-of-the-art CoT models including GPT o3-mini-high, Claude 3.7, and DeepSeek R1 achieve 0% accuracy on expert-level Sudoku puzzles that demand systematic backtracking [1, 7]. Analysis of scaling behaviour reveals that increasing Transformer width at fixed depth yields no improvement on Sudoku-Extreme, while increasing depth saturates quickly due to training instabilities [1]. These findings suggest that effective computational depth, rather than raw parameter count, is the primary bottleneck for complex reasoning.

3.2 HRM Architecture and Mechanisms

The HRM implements a recurrent architecture that addresses the computational depth limitations of fixed-depth Transformers while avoiding training instabilities common in Recurrent Neural Networks (RNNs) [14, 12], such as exploding and vanishing gradients in Backpropagation Through Time (BPTT) [1, 15].

3.2.1 Core Design Principles

The HRM's architecture is inspired by three principles observed in mammalian cortical processing [1, 16]. First, *hierarchical processing* organises computation across multiple levels, where higher-level areas integrate information over extended timescales while lower-level areas handle immediate, detailed processing. Second, *temporal separation* establishes distinct operational timescales analogous to neural rhythms: slow theta waves (4–8 Hz) for high-level integration and fast gamma oscillations (30–100 Hz) for local computation. Third, *recurrent connectivity* enables iterative refinement through feedback loops, allowing progressive error correction without the computational burden of BPTT [15, 17].

The model consists of four components: an input network that converts inputs into a working representation, a low-level recurrent module (L-module), a high-level recurrent module (H-module), and an output network that generates predictions. The computation unfolds over N high-level cycles, each consisting of T low-level timesteps, yielding $N \times T$ total computational steps. The L-module updates at every timestep conditioned on its previous state, the current H-module state, and the input representation, while the H-module updates only once per cycle using the converged state of the L-module [1].

3.2.2 Hierarchical Convergence

As described by Geng *et al.* [14], a challenge in recurrent architectures is premature convergence. As hidden states approach fixed points, update size diminishes, halting computation and limiting effective reasoning depth. Standard RNNs display this behaviour within tens of iterations.

HRM addresses this through hierarchical convergence. During each high-level cycle, the L-module performs T iterative updates, converging to a local equilibrium determined by the current H-module state. Upon convergence, the H-module incorporates the L-module's final state and performs a single update, establishing a new context that effectively resets the L-module's computational space and initiates convergence toward a different equilibrium in the subsequent cycle.

This hierarchical structure yields effective computational depth of $N \times T$ rather than T typical of standard RNNs. Empirical analysis demonstrates this advantage: as shown in Figure 3.1, forward residuals in standard RNNs decay toward zero within 20–30 steps, while HRM maintains computational activity for 200+ steps [1].

3.2.3 Training Methodology: One-Step Gradient Approximation

Recurrent networks usually require BPTT for gradient computation, consuming $O(T)$ memory to store hidden states. HRM avoids this through One-Step Gradient Approximation (OSGA), achieving $O(1)$ memory complexity [1], building on Deep Equilibrium Model (DEQ) theory explored by Bai *et al.* [10]. In practice, HRM computes gradients only through the final states of each module, ignoring all other timesteps. The model also incorporates deep supervision, applying loss feedback at multiple stages during training. This is critical for making training feasible on a single consumer GPU. Figure 3.2 illustrates the architecture and the OSGA training approach.

However, recent analysis by Ge *et al.* [8] identifies a discrepancy: while Adaptive Computation Time (ACT) [11] learns halting policies during training, the original HRM evaluation always executes maximum reasoning steps, ignoring the learned policy. Additionally, Ge *et al.* observe that OSGA is functionally equivalent to training paradigms used

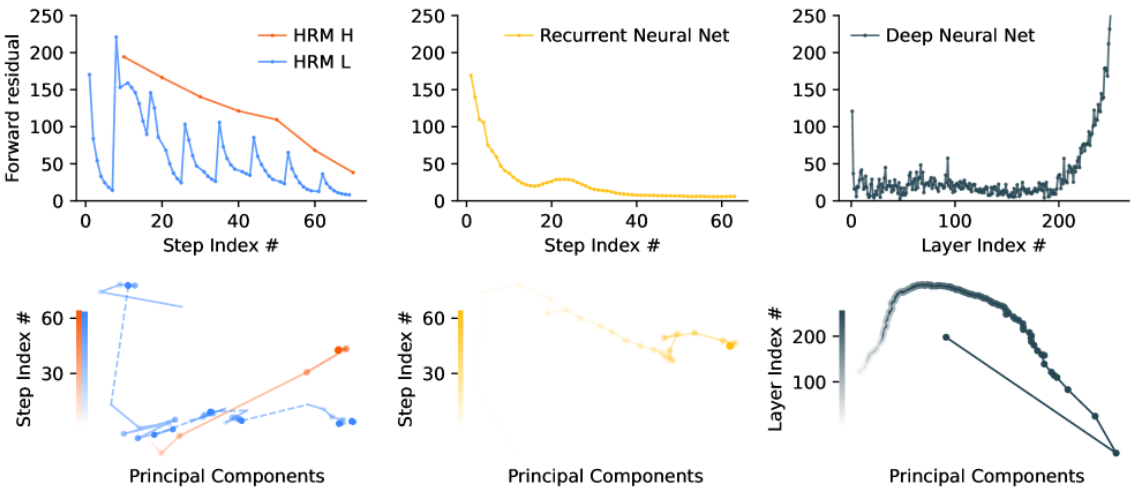


Figure 3.1: Forward residual comparison across architectures. Left: HRM H-module (red) and L-module (blue) maintain sustained activity across hundreds of steps. Middle: Standard RNN with rapid residual decay within tens of steps. Right: Deep Neural Network with fixed depth. Reproduced from Wang *et al.* [1].

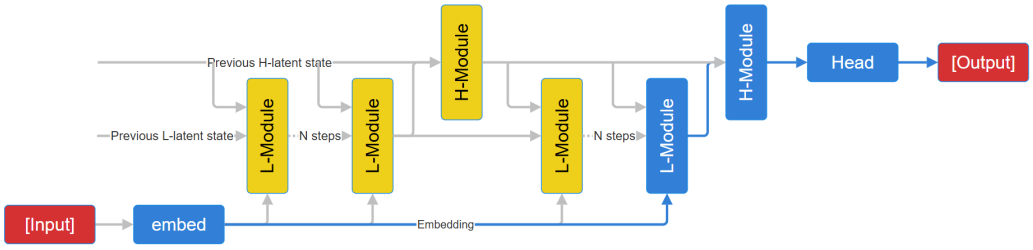


Figure 3.2: The HRM dual-module architecture as described by Wang *et al.* [1]. Modules coloured in blue experience One-Step Gradient Approximation during training, reducing memory requirements from $O(n)$ with BPTT to $O(1)$.

in diffusion models [8], suggesting HRM may be better understood as a deep feed-forward network rather than a genuinely recurrent architecture.

3.3 Performance and Benchmarks

The HRM achieves notable performance on three challenging reasoning benchmarks [1]. On ARC-AGI-1, HRM attains 40.3% accuracy, surpassing o3-mini-high (34.5%), Claude 3.7 8K (21.2%), and DeepSeek R1 (21.0%). On Sudoku-Extreme, HRM achieves 55.0% accuracy while all CoT-based models score 0%. On Maze-Hard, HRM reaches 74.5% while tested CoT models fail entirely. These results are striking given that HRM operates with 27 million parameters while compared models have hundreds of billions.

Table 3.1: Comparison of HRM and Chain-of-Thought LLMs across key dimensions.

Dimension	**HRM**	**CoT LLMs**
Parameters	27M	300B–2T
Training Data	1,000 examples	Tens of trillions of tokens
Sudoku-Extreme	55%	0%
ARC-AGI-1	40.3%	34.5% (best)
Maze-Hard	74.5%	0%
Memory Complexity	$O(1)$	$O(L)$

3.4 Memory Patterns and Computational Cost

HRMs exhibit constant memory consumption regardless of reasoning depth, a key advantage over Transformers where memory scales linearly with sequence length [10]. While Transformers may exhaust device memory on extended reasoning chains, HRM maintains predictable consumption enabling reliable deployment without memory leaks. With only 27 million parameters, the HRM occupies approximately 108 MB in 32-bit floating-point representation, fitting comfortably within the memory constraints of edge devices such as Raspberry Pi 5 with 16 GB RAM [1].

However, the dual-module architecture introduces memory overhead compared to single-module recurrent networks. The H-module and L-module each maintain separate hidden state representations, doubling the base memory footprint relative to equivalent single-module architectures [1]. Inter-module communication at each high-level cycle, transferring converged low-level states and broadcasting updated high-level context, may introduce performance bottlenecks on ARM platforms with limited memory bandwidth. The L-module-only variant eliminates this overhead entirely, which is relevant for the edge deployment goals of this project.

3.5 Biological Correspondence

The HRM demonstrates parallels with principles of brain function. Neuroscientific research suggests that the function of a brain region is related to its structure [18, 19]. Higher-order areas responsible for complex reasoning exhibit higher activity, reflecting their capacity to represent task-relevant variables. This gives rise to a dimensionality hierarchy: effective dimensionality increases from low-level sensory areas to high-level associative cortex [20].

The HRM reproduces this organisation. Analysis using Participation Ratio (PR) reveals that the high-level module operates in higher-dimensional space (PR = 89.95) compared to the low-level module (PR = 30.22) when trained on Sudoku-Extreme [1]. This ratio closely matches the approximately 2.25:1 ratio observed between high-level and low-level areas in mouse cortex [20]. Importantly, this hierarchical organisation is an emergent property of training rather than an engineered feature: analysis of an untrained HRM shows no separation of dimensionality, suggesting that the learned separation arises from task demands.

This biological correspondence is relevant to the critical question of whether the H-module is necessary. If the dimensionality separation emerges from task pressure, then

a single-module variant with sufficient capacity might develop its own internal hierarchy through training, achieving comparable performance without explicit architectural separation.

3.6 Critical Perspectives

The HRM offers distinct advantages for definitive reasoning tasks. Data efficiency reduces training requirements by three orders of magnitude compared to CoT-based approaches: the model achieves strong performance with 1,000 input-output pairs per task, a result of learning in continuous latent space rather than linear token generation [21]. The compact architecture (27M parameters, approximately 108 MB) enables deployment on resource-constrained hardware. Resistance to partial errors through holistic iterative refinement means that errors in latent states can be corrected in later iterations, unlike CoT where an incorrect step propagates through the remainder of the chain [4, 5]. The recurrent architecture also achieves a form of Turing-completeness given sufficient computational resources, unlike fixed-depth Transformers constrained to AC^0/TC^0 complexity classes [1].

However, HRM has limitations. Task scope is the most fundamental constraint: current implementations excel on two-dimensional grid-based reasoning but do not handle natural language tasks, open-ended generation, or domains requiring world knowledge. Interpretability presents another challenge: while CoT reasoning produces human-readable steps, HRM operates entirely in latent space, making it difficult to understand why the model produces particular solutions. Inference cost also merits consideration: solving complex problems may require hundreds of computational steps, and for problems where CoT succeeds, token-by-token generation may be more efficient.

Most significantly, ablation studies by Ge *et al.* [8] suggest the high-level module may provide minimal benefits: an HRM variant using only the L-module achieves comparable accuracy to the full system. This finding, confirmed by the ARC Foundation [9], raises questions about whether hierarchy is necessary or whether computational depth alone drives performance. The L-module-only variant also trains approximately 2.4× faster than the full HRM [8], which is significant for resource-constrained research environments. This project investigates this question directly.

3.7 Cross-Platform Inference Frameworks

Deploying HRMs across heterogeneous edge hardware requires intermediate representations that preserve model semantics while enabling efficient execution on diverse processor architectures.

3.7.1 ONNX Representation Challenges

The Open Neural Network Exchange (ONNX) format has emerged as the de facto standard for platform-agnostic model representation, defining a comprehensive operator set and computation graph structure that enables models trained in PyTorch or TensorFlow to execute across diverse inference engines [2, 22]. Figure 3.3 illustrates the standard ONNX deployment pipeline from training framework to edge device.

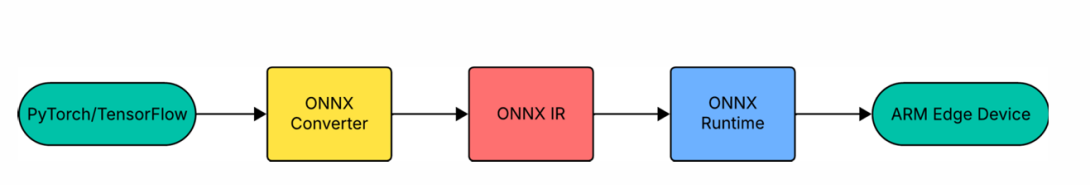


Figure 3.3: ONNX cross-platform deployment workflow: a model trained in PyTorch is converted to the ONNX intermediate representation and executed via ONNX Runtime on the target device. Adapted from Pochelu [2].

However, HRM characteristics present unique challenges. The iterative refinement process involves dynamic computation graphs where iteration counts vary based on problem complexity [23]. While ONNX supports recurrent operators through loop constructs, fixed-point iteration semantics require careful modelling to preserve convergence properties during conversion. Jiang *et al.* conducted an extensive analysis of ONNX converter failures, finding that approximately 75% of defects occur during node conversion and 33% of reported failures result in semantically incorrect models [24]. These findings raise significant concerns for HRM deployment, where convergence behaviour depends on precise numerical semantics.

3.7.2 Runtime Optimisation Applicability

ONNX Runtime provides optimisation techniques including operator fusion, constant folding, and memory layout transformations [25, 26]. However, these optimisations were designed primarily for CNNs and Transformers rather than iterative reasoning models. Operator fusion strategies that merge consecutive layers become difficult when the same operators execute repeatedly with varying inputs, and static memory allocation based on predetermined layer sequences becomes problematic when iteration counts vary dynamically [23]. Table 3.2 summarises the applicability across architectures.

Table 3.2: ONNX Runtime optimisation applicability across architectures.

Optimisation	**CNNs**	**Transformers**	**HRMs**
Operator Fusion	High	High	Limited
Memory Planning	Static	Static	Dynamic
ARM NEON	Optimised	Optimised	Untested

ARM NEON SIMD extensions on Cortex-A76 processors (Raspberry Pi 5) offer vectorised operations achieving 2–4× speedups for neural inference [27], representing a useful optimisation opportunity. However, ONNX Runtime's ARM-specific optimisations primarily target inference patterns observed in classification and object detection networks [2]. The extent to which these optimisations benefit HRM's iterative refinement patterns remains an open empirical question, and the absence of HRM-specific deployment guidelines represents a significant gap in current tooling [22].

3.8 Edge Device Considerations

3.8.1 Raspberry Pi 5 Hardware

The Raspberry Pi 5 uses a Broadcom BCM2712 system-on-chip with four ARM Cortex-A76 cores running at 2.4 GHz. Each core includes a 64 KB L1 data cache and a 64 KB L1 instruction cache, with a shared 512 KB L2 cache per core and a 2 MB shared L3 cache. The 16 GB LPDDR4X memory provides approximately 34 GB/s bandwidth. For context, the HRM's 6.9M parameters in FP32 occupy roughly 27.6 MB, which fits comfortably within the available memory. The Cortex-A76 cores support ARM NEON SIMD extensions, enabling vectorised floating-point operations that are relevant for matrix multiplication workloads during inference.

The operating system used for deployment is Debian 13 (Trixie), 64-bit with an aarch64 kernel. ONNX Runtime ships prebuilt aarch64 wheels for this distribution, allowing installation via `pip install onnxruntime` without cross-compilation. Python 3.11 is the system default.

3.8.2 Quantisation for Iterative Architectures

Quantisation reduces numerical precision from 32-bit floating-point to lower bit-widths such as 16-bit or 8-bit integers, decreasing memory footprint and potentially accelerating inference [28, 29]. For edge deployment on ARM platforms, quantisation can be important for achieving acceptable latency within power budgets.

However, the interaction between reduced precision and fixed-point iteration convergence introduces complications absent in single-pass feedforward networks. In feedforward networks, quantisation errors are localised: each layer's rounding error only affects downstream layers. In iterative architectures such as HRM, errors accumulate across successive iterations, and for models performing dozens to hundreds of refinement cycles, this accumulation may destabilise convergence, cause oscillatory behaviour, or increase the iteration count required to reach acceptable solution quality [30]. Convergence guarantees depend on maintaining sufficient precision to preserve the contraction mapping properties of the recurrent operator [10]. Figure 3.4 illustrates this contrast between feedforward and iterative architectures.

Mixed-precision strategies offer a potential compromise by allocating higher precision to convergence-critical operations while applying aggressive quantisation to less sensitive components [30]. For HRMs specifically, the L-module's rapid iterative refinement may require higher precision to maintain convergence stability, while less frequently updated components may tolerate lower precision. However, identifying the optimal precision allocation requires extensive empirical evaluation on target hardware. This project uses FP32 throughout, leaving quantisation exploration as future work.

3.8.3 ARM-Specific Optimisation Challenges

ARM processors dominate the edge computing landscape, with platforms such as Raspberry Pi 5 employing ARM Cortex-A76 cores featuring NEON SIMD instruction sets. These are capable of performing vectorised operations on multiple data elements simultaneously, offering 2–4× speedups for conventional neural network inference [27]. Effective use of these capabilities improves inference throughput, but vectorisation for recurrent

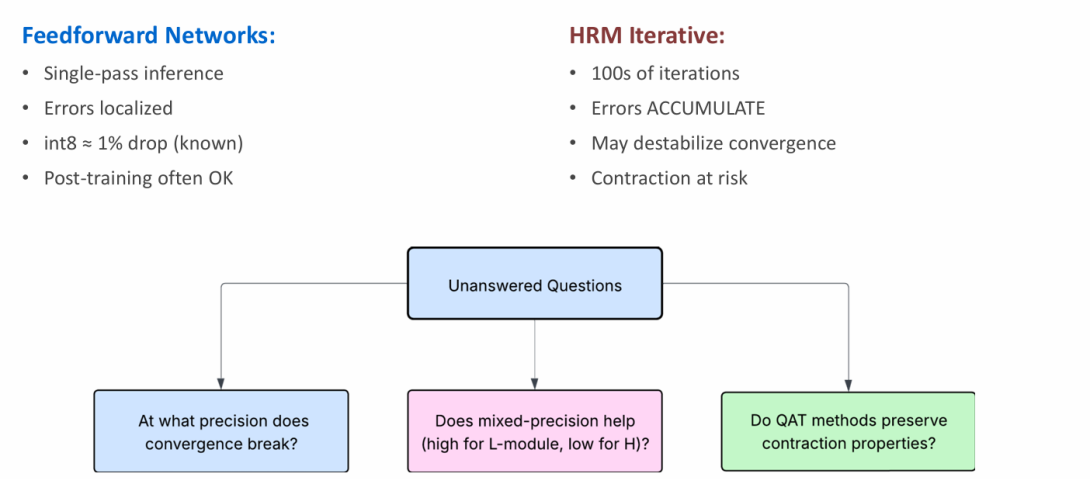


Figure 3.4: Quantisation effects on feedforward versus iterative architectures. In feedforward networks, rounding errors are localised and well characterised. In iterative architectures such as HRM, errors accumulate across reasoning steps and may destabilise convergence. Three open questions remain: at what precision does convergence break, whether mixed-precision strategies help, and whether quantisation-aware training preserves contraction properties.

architectures proves more challenging than for feedforward networks due to data dependencies between successive iterations that constrain parallelisation.

HRM computation introduces memory access patterns that may poorly match ARM cache hierarchies optimised for sequential or strided access. At each iteration, the recurrent operator must load hidden states from memory, perform matrix computations, and write updated states back. On ARM platforms with limited cache capacity and moderate memory bandwidth, these repeated memory transfers may dominate computational costs [31]. Cache-aware scheduling strategies that maximise temporal locality, keeping frequently accessed hidden states resident in cache across iterations, are important for minimising memory traffic.

Recent benchmarking by Wang *et al.* [27] demonstrates that memory bandwidth limitations often constrain achievable parallelism for neural network workloads on edge platforms. For the L-module-only variant with 6.9M parameters (approximately 27.6 MB in FP32), the model fits comfortably within the Pi 5's L3 cache and main memory, but the repeated iteration pattern means that the same weights are accessed 16 times during a single forward pass. Whether ONNX Runtime's execution provider effectively exploits this reuse pattern on ARM hardware is an empirical question that this project addresses through benchmarking.

3.9 Research Gaps and Summary

Table 3.3 summarises the critical research gaps identified across the domains examined in this review. As illustrated in Figure 3.5, no existing research bridges all three domains for HRM edge deployment, leaving practitioners without guidance for this deployment scenario.

This project addresses several of these gaps directly. The L-module-only variant tests whether hierarchical structure is necessary, the Weighted Maze task extends evaluation

Table 3.3: Critical research gaps in HRM edge deployment.

Domain	**Research Gap**
HRM Architecture	No ARM hardware benchmarks for iterative reasoning architectures; unclear whether hierarchical structure is necessary
Cross-Platform	ONNX converter issues with dynamic control flow; no HRM-specific deployment guidelines
Edge Optimisation	Unknown quantisation effects on fixed-point convergence stability
Novel Tasks	HRM evaluated only on unweighted grid tasks; no cost-sensitive pathfinding benchmarks
Integration	No comprehensive Raspberry Pi or similar edge deployment studies for HRM

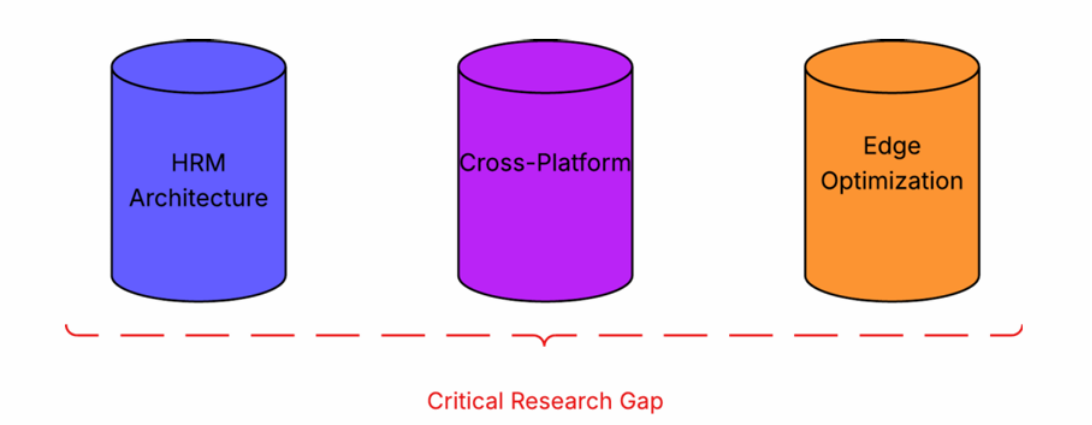


Figure 3.5: Synthesis of critical gaps across three domains: HRM architecture, cross-platform frameworks, and edge optimisation. Each domain has established strengths but significant unresolved gaps. No existing research bridges all three domains for HRM edge deployment.

to cost-sensitive pathfinding, the ONNX export pipeline resolves six specific conversion blockers for iterative reasoning models, and inference benchmarking on Raspberry Pi 5 provides the first empirical data on HRM performance on ARM-based edge hardware.

Chapter 4

System Design

This chapter describes the system architecture and design decisions made throughout the project. It covers the overall structure of the codebase, the architecture of both model variants, the encoding and generation of both reasoning tasks, the ONNX export pipeline that I designed and implemented, and the continuous integration infrastructure. The full HRM architecture and the training pipeline design are led by Fionn McCarthy, and are summarised here where they intersect with the components I built. The Weighted Maze generator and Sudoku generator are described here from a system design perspective; both were implemented by Fionn McCarthy.

4.1 Overall System Architecture

The project is structured as a Python package hosted at `https://github.com/fionntmcc/cross-platform-hrm`. The repository follows a modular design with clear separation between model definitions, dataset generation, training logic, and evaluation. Table 4.1 summarises the top-level directory structure.

Table 4.1: Repository directory structure.

Directory / File	**Purpose**
`hrm/`	Core package containing model definitions, dataset generators, training scripts, and utility functions
`scripts/`	Entry points for training, inference, dataset generation, ONNX export, and visualisation
`tests/`	Unit and integration tests for model components and data pipelines
`docs/`	Project documentation
`.github/workflows/`	GitHub Actions CI/CD pipeline
`requirements.txt`	Python dependencies

A deliberate design decision was made to avoid CUDA-specific extensions such as FlashAttention, which the original Sapient Inc. codebase requires [1]. PyTorch was chosen over TensorFlow because the official HRM reference implementation uses PyTorch, simplifying validation against the original codebase. ONNX was chosen over TensorRT (NVIDIA-only) or CoreML (Apple-only) because it is the only format that targets both

x86 and ARM platforms through a single export. This cross-platform compatibility is reflected in the repository name. Figure 4.1 illustrates the end-to-end data flow from dataset generation through deployment.

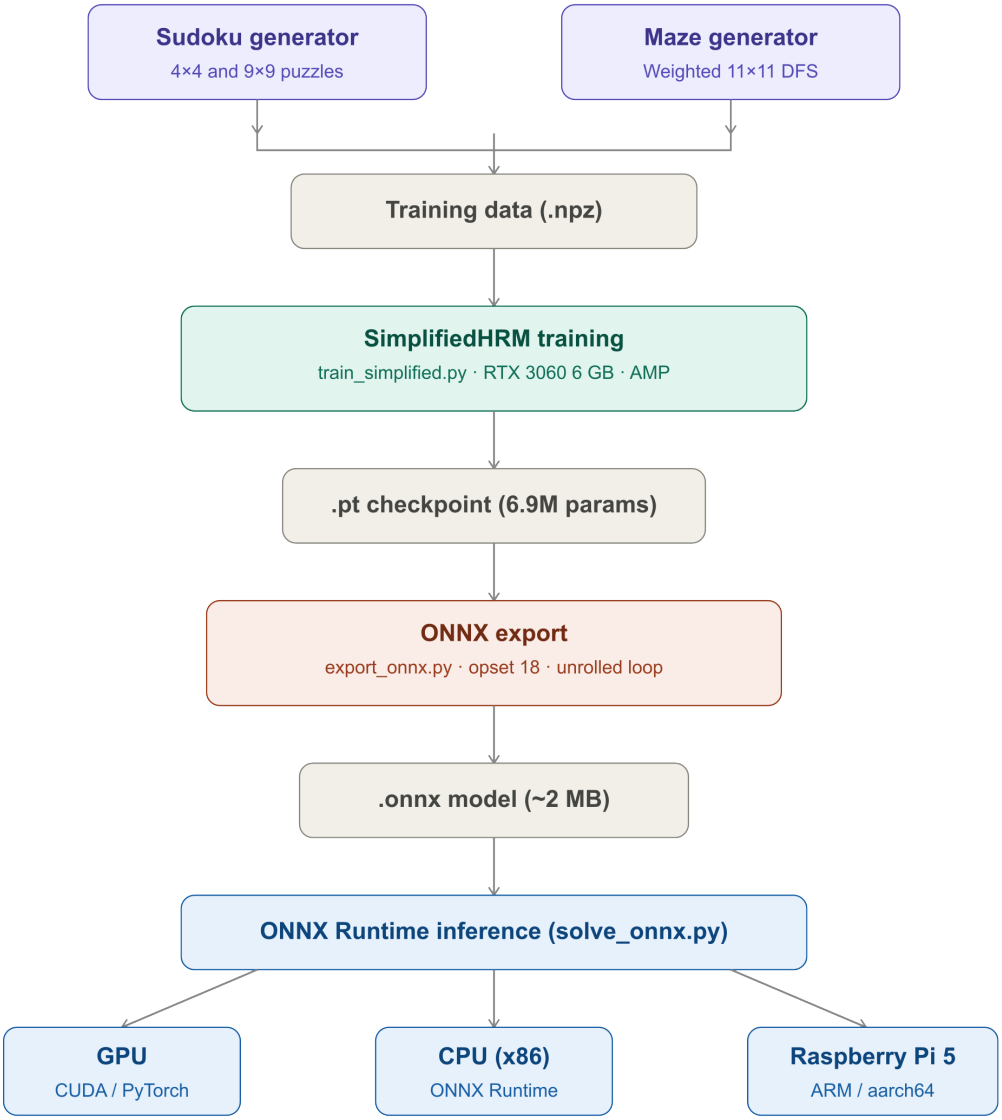


Figure 4.1: High-level system architecture showing the pipeline from dataset generation through model training to ONNX export and deployment across GPU, CPU, and Raspberry Pi 5 hardware targets.

4.2 Model Architecture: Full HRM

The full HRM implementation, led by Fionn McCarthy, follows the architecture described by Wang *et al.* [1]. The model consists of four networks: an input network, an L-module, an H-module, and an output network. The full HRM uses $N = 16$ high-level cycles of $T = 16$ low-level steps each, yielding 256 total computational steps per forward pass.

Both modules use multi-head self-attention with rotary positional embeddings (RoPE) and SwiGLU-activated MLPs. The full model has approximately 27 million parameters.

4.3 Model Architecture: L-Module Only (Simplified HRM)

The L-module-only variant, which I implemented from scratch, tests the hypothesis from Ge *et al.* [8] that hierarchical structure may be unnecessary for effective reasoning. This variant removes the H-module entirely while keeping the input network, L-module, and output network unchanged. Table 4.2 lists the configuration.

Table 4.2: Simplified HRM (L-module-only) configuration.

Parameter	**Value**
Total Parameters	≈6.9M
Hidden Dimension	256
Transformer Blocks	8 (weight-shared)
Attention Heads	8
Activation	SwiGLU
Positional Encoding	Rotary (RoPE)
Normalisation	RMSNorm
Reasoning Steps	16

Without the H-module, the L-module no longer receives periodic resets that redirect its computation toward new equilibria. Instead, it iterates for the full $N \times T$ steps in a single continuous sequence. The key architectural feature that distinguishes this from a simple feedforward network is *self-conditioning*: at each reasoning step, the model's current prediction is fed back as input to the next step, allowing iterative refinement. The following pseudocode illustrates the core loop:

```
1 h = input_embedding(puzzle)
2 for step in range(n_reasoning_steps):
3     h_context = h.detach()  # stop gradients
4     h = transformer_block(h, h_context)
5     prediction = output_head(h)
6     h = re_embed(prediction, puzzle)
```

Listing 4.1: Self-conditioning reasoning loop in the L-module-only variant.

The `detach()` call on line 3 is significant: it prevents gradients from flowing through the full iteration history, implementing the one-step gradient approximation described in Chapter 3. This keeps memory consumption constant ($O(1)$) regardless of the number of reasoning steps, which is what makes training feasible on a single consumer GPU.

The training procedure for this variant is identical to the full HRM: one-step gradient approximation with deep supervision and the same hyperparameters. Fionn McCarthy implemented the one-step backpropagation mechanism for this variant, adapting the gradient approximation to work without the H-module's periodic resets.

4.4 Task Implementations

Both tasks use integer token encodings mapped to learned embedding vectors. For Sudoku, tokens 0 (empty) and 1–9 (digits) encode the 9×9 grid. The output is a probability distribution over digits 1–9 for each cell. For the Weighted Maze, tokens 0 (wall), 1 (open path), 2 (start), 3 (goal), and 4–9 (weighted cells with movement cost equal to token value) encode the 15×15 grid. The output is a binary classification for each cell: on or off the optimal minimum-cost path, which is computed during dataset generation using Dijkstra's algorithm. Both generators were implemented by Fionn McCarthy; the Sudoku generator produces puzzles of configurable difficulty (easy through expert-level based on backtracking count) and the Weighted Maze generator produces grids with unique optimal paths.

4.5 Training Pipeline

The training pipeline is shared between both model variants and both tasks. It was designed by Fionn McCarthy and is summarised here because it directly affects the ONNX export and inference behaviour.

Training proceeds as follows. For each batch, the input grid is passed through the input network to produce an embedding. The recurrent module then performs 16 iterations of refinement (with or without the H-module depending on the variant). At several intermediate checkpoints during this iteration, the output network produces a prediction and the loss is computed against the ground truth using cross-entropy. The total loss is the sum of these intermediate losses (deep supervision). Gradients are computed using the one-step approximation, meaning that only the final state of each checkpoint contributes to the gradient computation, and the model parameters are updated using the Adam optimiser with cosine annealing learning rate schedule.

A training performance tracking system logs metrics to CSV across epochs and automatically generates loss and accuracy plots at the end of each training run. This infrastructure proved valuable for diagnosing training issues and producing the convergence curves presented in Chapter 5. Figure 4.2 illustrates the training loop.

4.6 ONNX Export Pipeline

I designed and implemented the ONNX export pipeline to enable the trained model to run on any platform without a PyTorch installation. The export script (`export_onnx.py`) converts a trained PyTorch checkpoint into an ONNX graph that can be executed by ONNX Runtime on x86, ARM, or any other supported architecture.

4.6.1 Conversion Challenges

Direct ONNX export of the Simplified HRM fails because the model uses six patterns that the ONNX tracer cannot represent. These are not bugs in the model but rather features of dynamic Python code that have no static ONNX equivalent. Table 4.3 lists each blocker and the resolution applied in the `SimplifiedHRMForONNX` wrapper class.

The wrapper class shares all weights with the original trained model (no duplication). It constructs a version of the model where all dynamic decisions have been resolved at

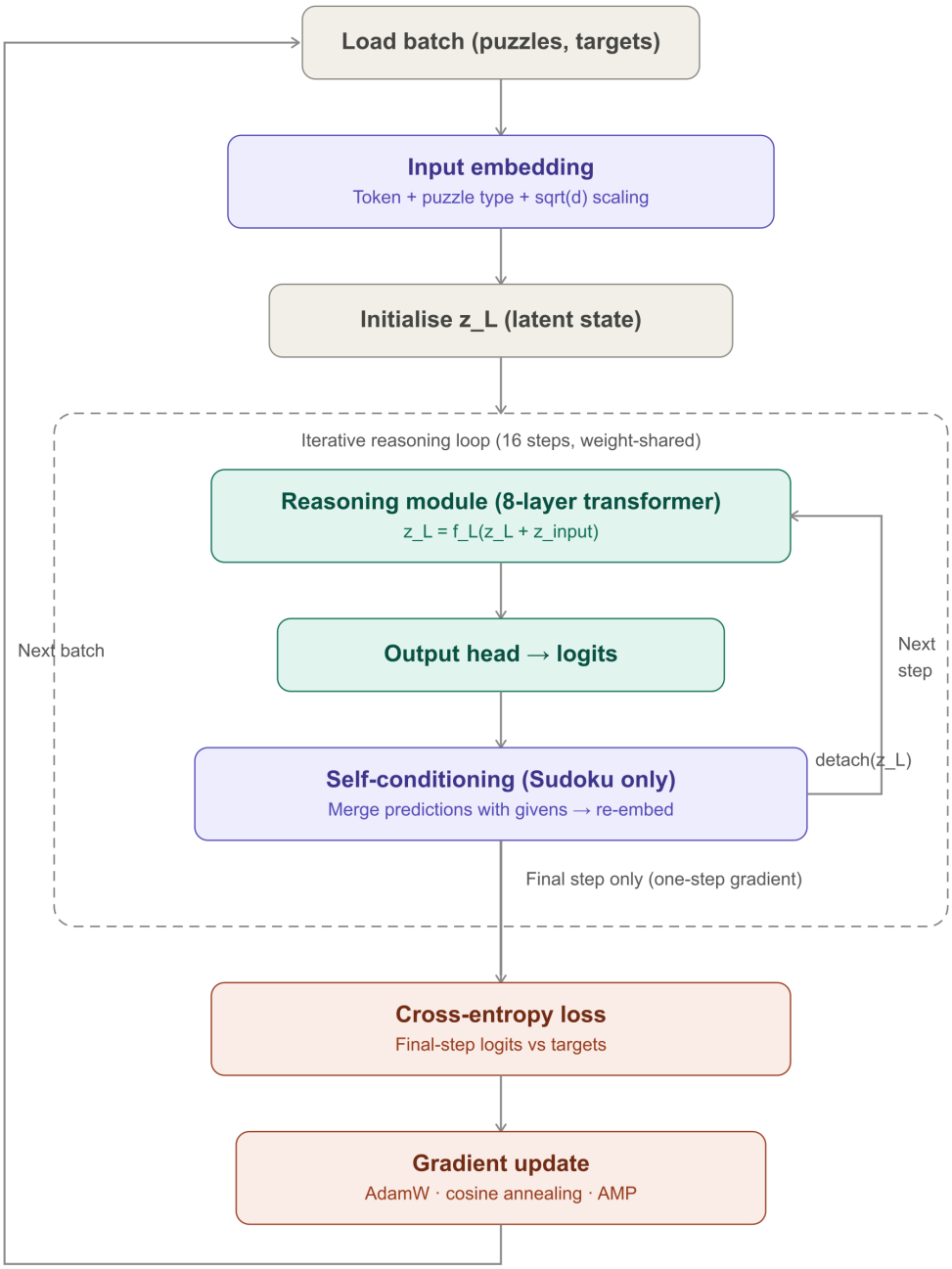


Figure 4.2: Training pipeline showing the flow from data loading through iterative refinement to gradient update using one-step gradient approximation with deep supervision.

construction time, producing a purely static computation graph that the ONNX tracer can follow.

4.6.2 Opset Selection

Initial attempts used ONNX opset 17, but this failed because the modern PyTorch dynamo exporter requires opset 18 for certain operator representations. Downconversion from opset 18 to 17 also failed due to missing operator support. The final pipeline uses opset 18

Table 4.3: ONNX conversion blockers and their resolutions in the wrapper class.

Blocker	Where	Resolution
PuzzleType enum dispatch	InputEmbedding, OutputHead	Baked at construction time
Python `for` loop with feedback	`forward()`	Unrolled as Python int
FlashAttention conditional	Attention module	Forced `_has_flash_attn = False`
CosSin tuple from RoPE	Reasoning step	Pre-computed as registered buffers
Enum-guarded embedding dispatch	InputEmbedding	Inlined to avoid enum branch
Dict return type	`forward()`	Returns predictions tensor only

exclusively, which is supported by ONNX Runtime 1.17+ on all target platforms including ARM.

4.6.3 Loop Unrolling and Graph Size

Because the Python reasoning loop is unrolled at export time, the ONNX graph contains 16 copies of the transformer block computation (one per reasoning step). The weights are shared (the same parameters are referenced 16 times), but the graph structure itself is duplicated. Each exported ONNX model is approximately 8.3–8.4 MB in size, consisting of an `.onnx` graph file and an accompanying `.onnx.data` external weight file. In total, the project produced eight model files: four PyTorch checkpoints (`.pt`) and four ONNX exports (`.onnx` plus `.onnx.data`), covering Sudoku 4×4, Sudoku 9×9, Maze 11×11, and Maze 15×15.

4.6.4 Numerical Verification

The export script automatically verifies that the ONNX model produces outputs numerically identical to the original PyTorch model on a set of test inputs. All three puzzle types (Sudoku 4×4, Sudoku 9×9, and Maze) were tested, with 5 out of 5 achieving exact numerical equivalence. This verification step is important because, as discussed in Chapter 3, Jiang *et al.* found that 33% of ONNX converter failures produce semantically incorrect models [24]. Figure 4.3 shows the full export pipeline.

4.7 Zero-Dependency Inference Script

The inference script (`solve_onnx.py`) is designed to run on any platform with only two Python packages: NumPy and ONNX Runtime. No PyTorch installation is required, which is important for the Raspberry Pi 5 deployment where PyTorch would consume significant storage and memory.

The script supports four modes of operation:

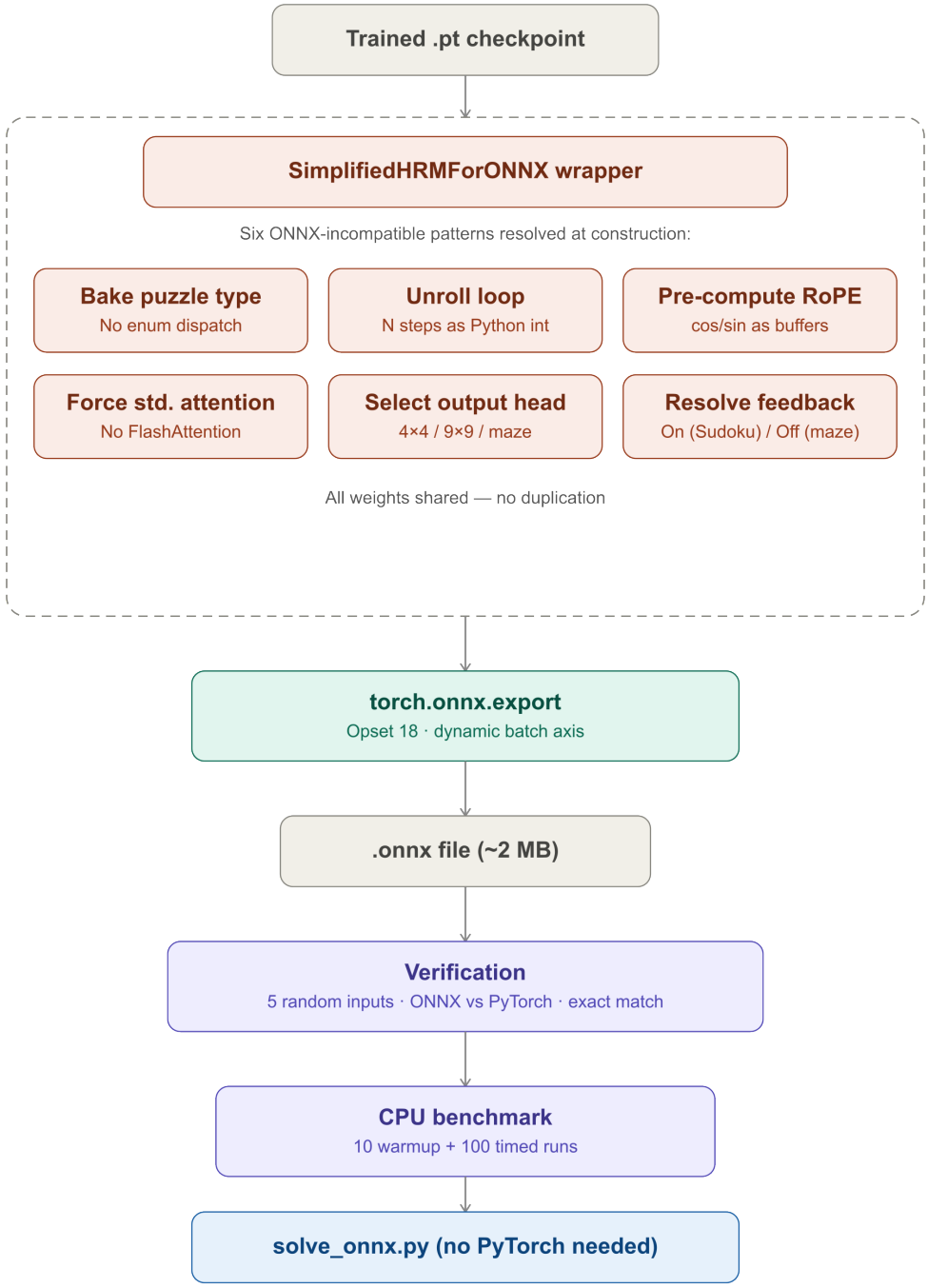


Figure 4.3: ONNX export pipeline showing the six transformations applied by the `SimplifiedHRMForONNX` wrapper, followed by export and numerical verification against the original PyTorch model.

- **Demo mode**: loads a trained ONNX model and runs inference on a small set of example puzzles, printing the predictions alongside the ground truth for visual inspection.

- **Dataset evaluation**: loads a dataset in `.npz` format and computes puzzle accuracy and cell accuracy across the full test set.

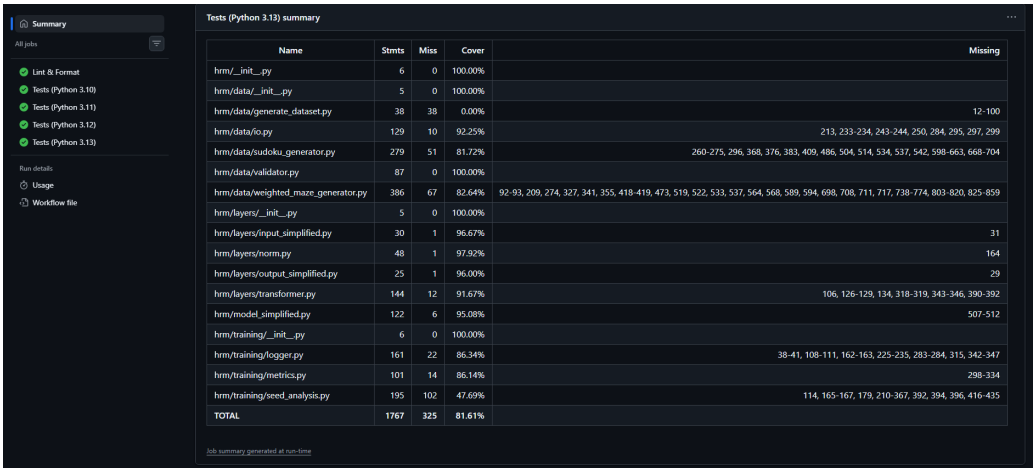
- **Benchmark mode**: runs repeated inference passes and reports average latency per puzzle, useful for comparing hardware platforms.
- **Interactive mode**: allows the user to input a puzzle manually and see the model's prediction.

For Weighted Maze evaluation, the script additionally computes path-specific metrics: path precision (fraction of predicted path cells that are actually on the optimal path), path recall (fraction of optimal path cells that the model correctly identifies), and path F1 score.

4.8 Continuous Integration Pipeline

I implemented the CI/CD pipeline using GitHub Actions. On every push to any branch, the pipeline runs three checks in sequence: Black formatting verification (ensuring all Python files conform to the Black code style), Ruff linting (catching common errors, import ordering issues, and style violations), and pytest execution (running the full test suite including unit tests for model components and integration tests for the data pipeline).

If any check fails, the pull request is blocked from merging. This was particularly important given that two developers were working on the same codebase; the pipeline caught several formatting inconsistencies and a handful of actual bugs during the project. Figure 4.4 shows the pipeline executing on a push event.



Name	Stmts	Miss	Cover	Missing
hrm/__init__.py	6	0	100.00%	
hrm/data/__init__.py	5	0	100.00%	
hrm/data/generate_dataset.py	38	38	0.00%	12-100
hrm/data/io.py	129	10	92.25%	213, 233-234, 243-244, 250, 284, 295, 297, 299
hrm/data/sudoku_generator.py	279	51	81.72%	260-275, 296, 368, 376, 383, 409, 486, 504, 514, 534, 537, 542, 598-663, 668-704
hrm/data/validator.py	87	0	100.00%	
hrm/data/weighted_maze_generator.py	386	67	82.64%	92-93, 209, 274, 327, 341, 355, 418-419, 473, 519, 522, 533, 537, 564, 568, 589, 594, 698, 708, 711, 717, 738-774, 803-820, 825-859
hrm/layers/__init__.py	5	0	100.00%	
hrm/layers/input_simplified.py	30	1	96.67%	31
hrm/layers/norm.py	48	1	97.92%	164
hrm/layers/output_simplified.py	25	1	96.00%	29
hrm/layers/transformer.py	144	12	91.67%	106, 126-129, 134, 318-319, 343-346, 390-392
hrm/model_simplified.py	122	6	95.08%	507-512
hrm/training/__init__.py	6	0	100.00%	
hrm/training/logger.py	161	22	86.34%	38-41, 108-111, 162-163, 225-235, 283-284, 315, 342-347
hrm/training/metrics.py	101	14	86.14%	298-334
hrm/training/seed_analysis.py	195	102	47.69%	114, 165-167, 179, 210-367, 392, 394, 396, 416-435
TOTAL	1767	325	81.61%	

Figure 4.4: GitHub Actions CI/CD pipeline running Black formatting, Ruff linting, and pytest checks on a push event. Failed checks block the pull request from merging.

4.9 Design Decisions and Trade-offs

Cross-platform compatibility over training speed. Avoiding CUDA-specific extensions ensures portability at the cost of training speed. This was a deliberate choice aligned with the project's goals.

Opset 18 over opset 17. The dynamo exporter's requirement for opset 18 was discovered empirically after opset 17 exports failed. This limits deployment to ONNX Runtime 1.17+, which is available on all target platforms but may not be on older systems.

Loop unrolling over ONNX Loop operator. The reasoning loop is unrolled at export time rather than using ONNX's built-in Loop operator. This produces a larger graph but avoids the convergence-sensitive Loop semantics discussed in Chapter 3, and simplifies debugging since the graph is entirely static.

FP32 throughout. No quantisation is applied at any stage. This provides a clean baseline for edge hardware performance and avoids the convergence stability concerns discussed in Section 3.8 of the technology review.

Chapter 5

System Evaluation

This chapter evaluates the project against the objectives set out in Chapter 1 and presents the results of training and benchmarking experiments. Results are reported for the L-module-only variant across both the Sudoku and Weighted Maze tasks. The full HRM has been implemented and is available on a separate branch of the repository, but benchmarking focused on the L-module-only variant for the reasons discussed in Section 5.8. All training and multi-seed analysis were conducted by Fionn McCarthy using the shared training pipeline; I led the ONNX export verification and edge deployment benchmarking.

5.1 Experimental Setup

Training experiments were conducted on Fionn McCarthy's GIGABYTE A16 PRO DXH laptop (Intel Core 7 240H, 10C/16T @ 5.2 GHz; NVIDIA RTX 5070 Ti 12 GB GDDR7; 32 GB LPDDR5; 1 TB Samsung NVMe; Windows 11) with mixed-precision training (AMP) enabled throughout. Inference was benchmarked on two platforms: my ASUS ROG Zephyrus M16 GU603ZM laptop (Intel i7-12700H, 14C/20T @ 4.7 GHz; RTX 3060 6 GB GDDR6; 16 GB DDR5-4800; 1 TB NVMe; Windows 11) running ONNX Runtime on CPU without GPU acceleration, and a Raspberry Pi 5 Model B Rev 1.1 (ARM Cortex-A76, 4C/4T @ 2.4 GHz; 16 GB LPDDR4X; 32 GB microSD) running Debian 13 (Trixie) with ONNX Runtime and no PyTorch dependency.

All models share the same configuration: 6,888,960 parameters, hidden dimension 256, 8 transformer layers, 16 weight-shared reasoning steps, SwiGLU activation, and Rotary Positional Embeddings (RoPE). ONNX models use opset 18, FP32 precision, and range from 8.3 to 8.4 MB per model. ONNX evaluation used 500 puzzles per task; PyTorch CPU evaluation used 200 puzzles per task. Sudoku evaluation uses the held-out eval set with mixed difficulty (seed 99). Maze evaluation uses unseen layouts generated with seed 99, producing grids the model never encountered during training. Fionn McCarthy's dissertation reports accuracy figures averaged across 10 independent training runs with 1,000 test puzzles per seed, which yield slightly different values (87.82% Sudoku puzzle accuracy, 75.20% Weighted Maze puzzle accuracy) due to the different evaluation protocol; the results reported here use single-seed ONNX inference on 500 puzzles, which is the protocol relevant to deployment verification and edge benchmarking. Table 5.1 summarises the training checkpoints produced.

Table 5.1: Training checkpoints produced on the RTX 5070 Ti 12 GB.

Model	Epochs	Val Acc	Training GPU
Sudoku 4×4	150	–	RTX 5070 Ti 12 GB
Sudoku 9×9	1,000	–	RTX 5070 Ti 12 GB
Maze 11×11	116	87.2%	RTX 5070 Ti 12 GB
Maze 15×15	187	66.8%	RTX 5070 Ti 12 GB

5.2 Sudoku Results

The L-module-only variant was trained on custom-generated Sudoku puzzles of mixed difficulty, spanning easy through expert-level. Two grid sizes were evaluated: 4×4 (used for rapid prototyping and architecture validation) and 9×9 (the standard benchmark size). Table 5.2 presents the evaluation results on 500 puzzles via ONNX Runtime, with accuracy identical across all three hardware platforms.

Table 5.2: L-module-only performance on Sudoku (n=500, ONNX inference).

Model	Dataset	Token Acc	Puzzle Acc
Sudoku 4×4	eval	92.8%	81.0%
Sudoku 9×9	eval	97.0%	83.2%

The 9×9 model achieves 83.2% puzzle accuracy on the evaluation set, meaning that 416 out of 500 test puzzles were solved with every cell correctly filled. Token accuracy reaches 97.0%, indicating that even on unsolved puzzles, the vast majority of individual cells are predicted correctly. The 4×4 model achieves 81.0% puzzle accuracy with 92.8% token accuracy; the lower token accuracy on the smaller grid reflects the proportionally larger impact of each incorrect cell in a 16-cell grid compared to an 81-cell grid. Figure 5.1 shows the training curves for the 9×9 model.

5.2.1 Analysis

The 83.2% puzzle accuracy demonstrates that the L-module-only variant learns effective Sudoku-solving behaviour from limited training data. For context, the original HRM paper reports 55% accuracy on Sudoku-Extreme using the full 27M-parameter two-module architecture [1]. Direct comparison requires care: the original evaluation focuses on the hardest puzzles (averaging 22 backtracks), whereas the dataset used here spans the full difficulty range from easy to expert. The result is best read as evidence that the simplified architecture retains meaningful Sudoku-solving capability across a representative difficulty distribution, consistent with Ge *et al.* [8].

The gap between token accuracy (97.0%) and puzzle accuracy (83.2%) reflects the strict evaluation criterion: a single incorrect cell out of 81 is sufficient to count the entire puzzle as failed. On average, unsolved puzzles contain only a small number of incorrect cells, indicating that the model comes close to a correct solution even when it fails.

A comparison between ONNX and PyTorch CPU inference is also informative. Running the same model via PyTorch on the laptop CPU (200 puzzles) yields 94.7% token accuracy and 83.0% puzzle accuracy for Sudoku 9×9. The near-identical puzzle accuracy (83.2% ONNX versus 83.0% PyTorch) confirms that the ONNX conversion does not degrade model quality, while the small difference in token accuracy (97.0% versus

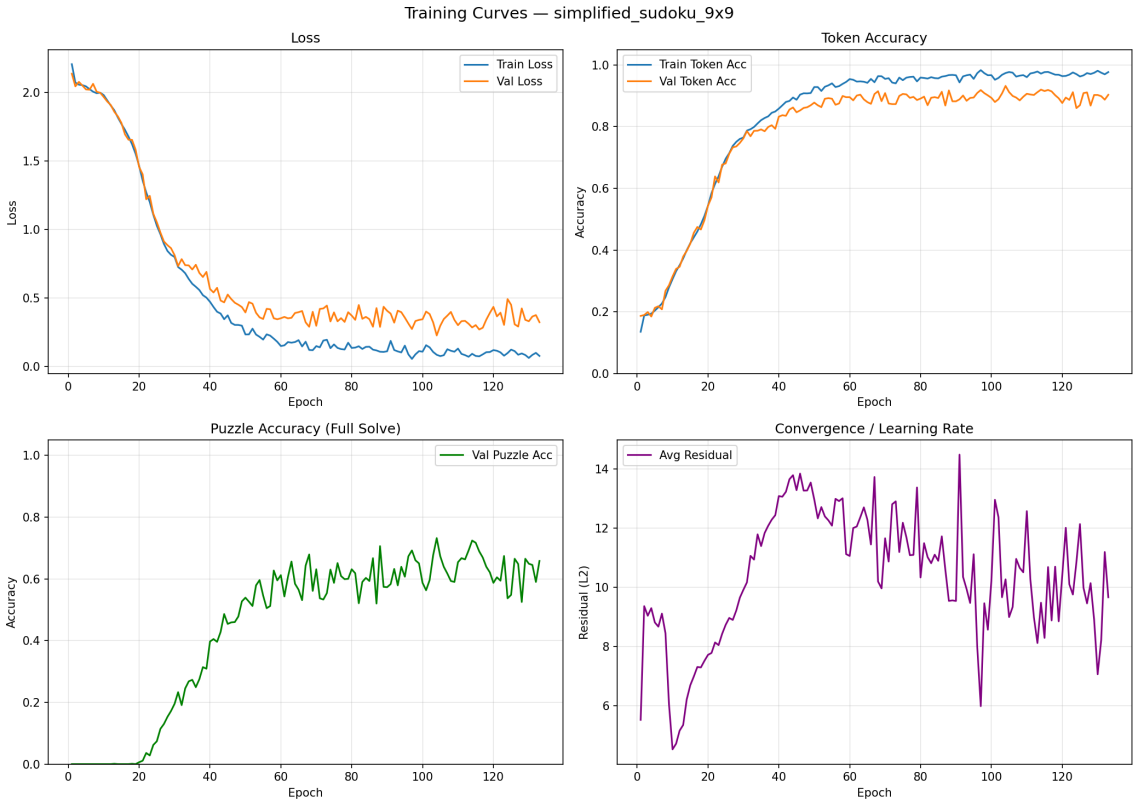


Figure 5.1: Sudoku 9×9 training curves. Top left: training and validation loss decreasing over epochs. Top right: token accuracy rising to approximately 97%. Bottom left: puzzle accuracy climbing over epochs. Bottom right: average L2 residual across reasoning steps.

94.7%) falls within the expected variance from different evaluation set sizes (500 versus 200 puzzles).

5.3 Weighted Maze Results

The L-module-only variant was trained on weighted mazes at two grid sizes: 11×11 and 15×15. Evaluation uses unseen maze layouts generated with a different random seed from training. Table 5.3 presents the results on 500 puzzles via ONNX Runtime, with accuracy identical across all three hardware platforms.

Table 5.3: L-module-only performance on Weighted Maze (n=500, ONNX inference, unseen layouts).

Model	**Token Acc**	**Puzzle Acc**	**Precision**	**Recall**	**Path F1**
Maze 11×11	98.9%	89.4%	96.9%	97.4%	97.2%
Maze 15×15	96.5%	65.4%	87.9%	87.5%	87.7%

The 11×11 model achieves 89.4% puzzle accuracy on unseen layouts, with 98.9% token accuracy and a path F1 score of 97.2%. The 15×15 model achieves 65.4% puzzle accuracy with 96.5% token accuracy and a path F1 of 87.7%. Both exceed the 60% target agreed upon as a sufficient benchmark, though the 15×15 result is closer to the boundary.

The path-specific metrics reveal that the model's predictions are spatially close to the ground truth even when they do not match exactly. On 11×11 mazes, 96.9% of cells the model predicts as on-path actually are on the optimal path (precision), and 97.4% of actual optimal path cells are correctly identified (recall). On 15×15 mazes, both precision and recall drop to approximately 87.5–87.9%, reflecting the increased difficulty of larger grids with more potential routes. Figure 5.2 shows the training curves for the Weighted Maze task.

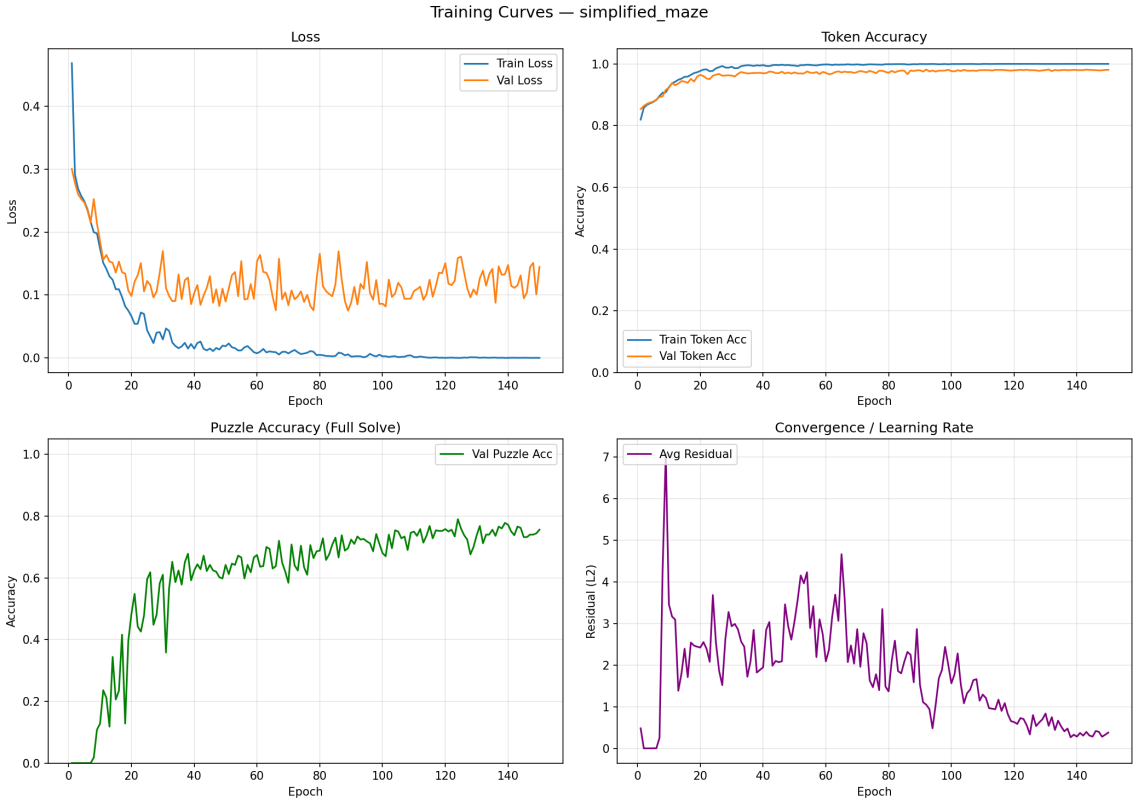


Figure 5.2: Weighted Maze training curves showing convergence over training.

5.3.1 Trajectory Visualisation

Figures 5.3 and 5.4 show the output of the trajectory visualiser (implemented by Fionn McCarthy), displaying the model's prediction at each of the 16 reasoning steps for a correctly solved and an incorrectly solved maze respectively.

In Figure 5.3, the initial prediction at Step 1 is a rough approximation with approximately 86% cell accuracy and 36% path accuracy, but by Step 5 the model converges to 100% on both metrics, with no further changes in subsequent steps. This convergence pattern supports the iterative refinement paradigm central to HRM.

In Figure 5.4, the model achieves high cell accuracy but selects a suboptimal route at a decision point where two paths have similar total cost, resulting in a failed exact-match evaluation despite the prediction being spatially close to the ground truth.

Figure 5.3: Reasoning trajectory for a correctly solved 15×15 maze across 16 iterative steps. The model converges to the correct solution by Step 5, with the bulk of computation occurring in the first four to five steps.

5.3.2 Analysis

The 65.4% puzzle accuracy on 15×15 Weighted Maze and 89.4% on 11×11 are notable results for tasks that have not been previously evaluated with any HRM variant. The original Maze-Hard benchmark uses uniform movement costs on 30×30 grids, where the full HRM achieves 74.5% [1]. Direct comparison is complicated by different grid sizes and the addition of weighted cells, but the results demonstrate that the L-module-only architecture can learn cost-sensitive pathfinding, extending the known capabilities of HRM-style reasoning beyond uniform-cost tasks. Both results exceed the 60% target.

The contrast between the two grid sizes is instructive: accuracy drops from 89.4% to 65.4% as the grid grows from 121 to 225 cells. This reflects the combinatorial difficulty of cost-sensitive pathfinding on larger grids, where more potential routes create more opportunities for the model to select a suboptimal path. The 15×15 grid also has a

Figure 5.4: Reasoning trajectory for an incorrectly solved 15×15 maze. Despite achieving high cell accuracy, the model selects a suboptimal route at a decision point where two paths have similar total cost.

proportionally lower path F1 (87.7% versus 97.2%), indicating that errors are not merely near-misses but involve more substantial deviations from the optimal route.

An important finding is the model's generalisation to unseen layouts. The maze evaluation uses layouts generated with seed 99, producing grids the model never encountered during training. The model's ability to achieve 89.4% puzzle accuracy on entirely new 11×11 layouts suggests genuine learning of cost-aware routing rather than pure memorisation. However, the drop to 65.4% on 15×15 unseen layouts suggests that generalisation becomes harder as grid complexity increases. Increasing the training set size, applying data augmentation (rotations, reflections), or training on a wider diversity of grid configurations are all potential mitigations.

The high token accuracy combined with lower puzzle accuracy also suggests that a post-processing step selecting the lowest-cost connected path from the model's soft

predictions could improve puzzle accuracy without retraining; this is left as future work.

5.4 ONNX Export Verification

The ONNX export was verified to produce outputs numerically identical to the original PyTorch model across all puzzle types. Table 5.4 summarises the verification results.

Table 5.4: ONNX numerical verification results. Accuracy is identical across all three platforms.

Puzzle Type	**Match**	**ONNX Size**	**CPU Latency**
Sudoku 4×4	5/5	8.4 MB	13.3 ms
Sudoku 9×9	5/5	8.4 MB	65.3 ms
Maze 11×11	5/5	8.3 MB	104.4 ms
Maze 15×15	5/5	8.3 MB	174.8 ms

All test inputs per puzzle type produced exact numerical equivalence between the PyTorch and ONNX outputs. The ONNX model sizes of 8.3–8.4 MB (consisting of the graph file and an external `.onnx.data` weight file) are consistent across puzzle types, confirming that the ONNX graph correctly shares weights across the 16 unrolled reasoning steps. This verification step is important because, as discussed in Chapter 3, Jiang *et al.* found that 33% of ONNX converter failures produce semantically incorrect models [24].

5.5 Edge Deployment on Raspberry Pi 5

The L-module-only variant was exported to ONNX format and deployed on a Raspberry Pi 5 Model B (16 GB LPDDR4X, Debian 13 Trixie, 64-bit) using ONNX Runtime with no PyTorch dependency. The deployment requires only two Python packages: `numpy` and `onnxruntime`, both of which install cleanly via `pip` on the Pi's default Python 3.11. No cross-compilation was needed. This simplicity is a direct benefit of the ONNX export approach: rather than porting the entire PyTorch runtime to ARM (which would consume over 2 GB of storage), only the lightweight ONNX Runtime inference engine is needed.

Accuracy on all four tasks remained **identical** across the training GPU, the laptop CPU via ONNX Runtime, and the Raspberry Pi 5 via ONNX Runtime. This confirms that the six conversion blockers resolved by the `SimplifiedHRMForONNX` wrapper (Table 4.3) successfully preserve the iterative reasoning semantics despite unrolling the computation loop into a static graph.

Inference latency was measured using the `solve_onnx.py` benchmark mode with 16 reasoning steps. Table 5.5 presents the results comparing the laptop CPU to the Raspberry Pi 5.

The Pi 5 is 4.9–5.8× slower than the i7-12700H, with the slowdown increasing at larger grid sizes. This pattern is consistent with cache pressure: the 6.9M-parameter model (27.6 MB in FP32) fits within the i7's larger caches, but the repeated 16-step iteration pattern places more pressure on the Pi 5's 2 MB L2 cache. All four tasks achieve sub-second latency on the Pi 5 except Maze 15×15 at 1.02 seconds. The Pi 5 runs in FP32 without quantisation, confirming that HRM-style iterative reasoning is feasible on modest ARM hardware.

Table 5.5: Inference latency per puzzle: laptop (i7-12700H) versus Raspberry Pi 5 (Cortex-A76).

Model	Cells	i7-12700H	Pi 5	Slowdown	Pi Thru
Sudoku 4×4	16	13.3 ms	65.2 ms	4.9×	15/s
Sudoku 9×9	81	65.3 ms	343.9 ms	5.3×	3/s
Maze 11×11	121	104.4 ms	521.7 ms	5.0×	2/s
Maze 15×15	225	174.8 ms	1,022.1 ms	5.8×	1/s

5.6 Cross-Task Comparison

Table 5.6 summarises the L-module-only variant's performance across all four tasks.

Table 5.6: L-module-only performance summary across all tasks (n=500, ONNX).

Model	Token	Puzzle	Path F1	CPU (ms)	Pi 5 (ms)
Sudoku 4×4	92.8%	81.0%	–	13.3	65.2
Sudoku 9×9	97.0%	83.2%	–	65.3	343.9
Maze 11×11	98.9%	89.4%	97.2%	104.4	521.7
Maze 15×15	96.5%	65.4%	87.7%	174.8	1,022.1

The model achieves higher puzzle accuracy on Maze 11×11 (89.4%) than on Sudoku 9×9 (83.2%), despite the maze having 50% more cells (121 versus 81). This may reflect the binary nature of the maze output (on-path or off-path, two classes) versus Sudoku (digits 1–9, ten classes including empty). The Maze 15×15 result (65.4%) is the lowest, reflecting the greater combinatorial difficulty of cost-sensitive pathfinding on larger grids. Latency scales approximately linearly with grid area across all four models, which is expected given that the transformer's self-attention operates over a sequence of cells.

5.7 Evaluation Against Objectives

Objective 1: Implement a functional HRM. Achieved. The full HRM with both H-module and L-module has been implemented and verified to train and produce predictions.

Objective 2: Implement an L-module-only variant. Achieved. The Simplified HRM variant with 6.9M parameters was implemented from scratch, preserving weight-shared transformer blocks, SwiGLU activation, and rotary positional embeddings.

Objective 3: Train and benchmark on Sudoku. Achieved. The variant achieved 83.2% puzzle accuracy and 97.0% token accuracy on 9×9 Sudoku (n=500), with a 4×4 prototype also trained and evaluated at 81.0% puzzle accuracy.

Objective 4: Train and benchmark on Weighted Maze. Achieved. The variant achieved 89.4% puzzle accuracy on 11×11 and 65.4% on 15×15 unseen layouts, both exceeding the 60% target. Path F1 scores of 97.2% and 87.7% demonstrate strong spatial reasoning.

Objective 5: ONNX export and edge deployment. Achieved. Eight model files produced (four `.pt` + four `.onnx`), all six conversion blockers resolved, numerical equivalence verified across all four puzzle types, and deployed on a Raspberry Pi 5 via

ONNX Runtime with zero PyTorch dependency. Accuracy is identical across all three platforms. Inference latency ranges from 65.2 ms (Sudoku 4×4) to 1,022.1 ms (Maze 15×15) on the Pi 5.

Objective 6: Continuous integration. Achieved. GitHub Actions pipeline enforces Black, Ruff, and pytest on every commit. The pipeline caught formatting inconsistencies and bugs throughout the project.

5.8 Discussion

5.8.1 Why the L-Module Only Was Prioritised

The decision to focus on the L-module-only variant was guided by two factors. The literature evidence from Ge *et al.* [8] indicates that it achieves comparable accuracy while training significantly faster, and given limited GPU resources, prioritising it allowed more thorough experimentation. Its reduced parameter count (6.9M versus 27M) and simpler forward pass also make it better suited for the edge deployment goals of the project.

5.8.2 Strengths

The project demonstrates several strengths. The ONNX export pipeline resolves six specific conversion blockers for iterative reasoning models, providing a reusable pattern for anyone needing to deploy fixed-point iteration architectures via ONNX. The numerical verification process confirms that the export preserves model semantics, addressing the concern from Jiang *et al.* [24] that a third of ONNX conversions produce semantically incorrect models. The same codebase and model ran on GPU, CPU, and ARM without modification, validating the cross-platform design philosophy. The Raspberry Pi 5 deployment provides the first empirical latency benchmarks for HRM-style inference on edge hardware, filling a gap identified in the technology review. The Weighted Maze results provide the first evidence that iterative latent reasoning can handle heterogeneous traversal costs. The L-module-only results on both tasks are consistent with Ge *et al.*'s finding that hierarchical structure may be unnecessary for effective latent reasoning [8], adding further evidence from an independent reimplementation.

5.8.3 Limitations

Several limitations should be acknowledged. The absence of full HRM benchmarks prevents the direct architectural comparison that formed part of the original objectives. The Sudoku evaluation uses a custom-generated mixed-difficulty dataset rather than the published Sudoku-Extreme corpus, which means that comparison with published results requires care. No quantisation was applied, so the interaction between reduced precision and convergence stability remains untested. Individual run variance is not reported with confidence intervals. Training time was not formally recorded, preventing empirical confirmation of the 2.4× speedup reported by Ge *et al.* [8].

5.8.4 Threats to Validity

The evaluation protocol uses a single seed (seed 99) for both Sudoku and Maze evaluation sets, rather than averaging across multiple seeds. The custom Sudoku dataset was

designed to match the Sudoku-Extreme difficulty distribution but has not been independently validated against it. The Raspberry Pi 5 latency figures may vary with ambient temperature (thermal throttling), background processes, and ONNX Runtime version. The ONNX and PyTorch evaluation set sizes differ (500 versus 200 puzzles), which may account for small accuracy differences between the two runtimes.

Chapter 6

Conclusion

This chapter summarises the project, its findings, and its contributions. It revisits the objectives established in Chapter 1, reflects on the strengths and limitations of the work, and identifies directions for future research.

6.1 Summary

This project implemented, extended, and deployed the Hierarchical Reasoning Model architecture introduced by Wang *et al.* [1]. Working as a pair, Fionn McCarthy and Kyrylo Kozlovskyi developed an independent, cross-platform HRM implementation in PyTorch, motivated by the architecture's performance on constraint-satisfaction tasks and by evidence from Ge *et al.* [8] that a simplified L-module-only variant may suffice.

My individual contributions centred on the L-module-only variant implementation, the CI/CD pipeline, the ONNX export pipeline with its six conversion blocker resolutions, the zero-dependency inference script, and the Raspberry Pi 5 edge deployment and benchmarking. Fionn McCarthy led the full HRM implementation, the Weighted Maze and Sudoku dataset generators, the training pipeline, and the multi-seed evaluation analysis.

6.2 Key Findings

The L-module-only variant achieved 83.2% puzzle accuracy and 97.0% token accuracy on mixed-difficulty 9×9 Sudoku (n=500 via ONNX Runtime). The results support the view that computational depth through iterative refinement, rather than hierarchical structure, is the primary driver of reasoning performance for these tasks, consistent with Ge *et al.* [8].

On the Weighted Maze task, the model achieved 89.4% puzzle accuracy on unseen 11×11 layouts (path F1 = 97.2%) and 65.4% on unseen 15×15 layouts (path F1 = 87.7%), both exceeding the 60% target. These represent the first evaluation of any HRM variant on cost-sensitive pathfinding tasks, extending the known capabilities of iterative latent reasoning beyond uniform-cost domains.

The ONNX export pipeline successfully resolved six specific incompatibilities between the dynamic Python model and the static ONNX graph format. Numerical verification confirmed exact equivalence across all four puzzle types. Models trained on the RTX 5070 Ti deployed without modification to both the i7-12700H laptop and the Raspberry Pi 5, with **identical accuracy** across all three platforms. On the Pi 5, inference latency

ranged from 65.2 ms (Sudoku 4×4) to 1,022.1 ms (Maze 15×15), with the Pi 5 being 4.9–5.8× slower than the laptop CPU. All tasks achieve sub-second latency on the Pi 5 except Maze 15×15. The Pi 5 runs in FP32 without quantisation via ONNX Runtime, with zero PyTorch dependency.

6.3 Objectives Revisited

Objective 1: Implement a functional HRM. Achieved. The full two-module HRM was implemented and verified.

Objective 2: Implement an L-module-only variant. Achieved. The Simplified HRM with 6.9M parameters was implemented from scratch with self-conditioning feedback.

Objective 3: Train and benchmark on Sudoku. Achieved. 83.2% puzzle accuracy, 97.0% token accuracy on 9×9 (n=500, ONNX).

Objective 4: Train and benchmark on Weighted Maze. Achieved. 89.4% puzzle accuracy on 11×11 and 65.4% on 15×15 unseen layouts, both exceeding the 60% target.

Objective 5: ONNX export and edge deployment. Achieved. Eight model files (four `.pt` + four `.onnx`), six conversion blockers resolved, deployed on Raspberry Pi 5 with identical accuracy and sub-second latency on three of four tasks.

Objective 6: Continuous integration. Achieved. GitHub Actions pipeline with Black, Ruff, and pytest enforced on every commit.

6.4 Unexpected Insights

Two findings emerged that were not anticipated at the project's outset. First, the architecture correction described in Section 2.2.2 revealed that the distinction between weight-shared iterative application and sequentially stacked modules is subtle but critical. The original prototype used feedforward stacking with HRM-inspired naming, which collapsed the intended fixed-point iteration into a different computation entirely. This experience underscored the importance of precise architectural understanding when reproducing recent research, particularly when the primary sources use terminology that can be interpreted in multiple ways.

Second, the maze generalisation results were more nuanced than initially expected. The model achieved 89.4% puzzle accuracy on unseen 11×11 layouts, demonstrating genuine cost-aware routing rather than pure memorisation of training layouts. However, this dropped to 65.4% on 15×15 unseen grids, suggesting that generalisation becomes harder as grid complexity increases. The trajectory visualisation revealed that the model builds solutions through iterative spatial refinement rather than anything resembling backtracking or search, converging to correct solutions by Step 5 of 16 in most cases. This process is closer to how diffusion models denoise images than to how Dijkstra's algorithm explores a graph.

6.5 Limitations

The most significant limitation is the absence of full HRM benchmarks, which prevents a direct architectural comparison. The custom Sudoku dataset and the 15×15 maze

grids mean that comparisons with published results on Sudoku-Extreme (expert-only) and Maze-Hard (30×30) require care. No quantisation was explored, training time was not formally recorded, and individual run variance is not reported with confidence intervals.

6.6 Future Work

Several directions for future research emerge from this project.

Complete the architectural comparison. Train and benchmark the full HRM on both tasks under identical conditions to the L-module-only variant, providing a controlled answer to whether hierarchical structure adds value.

Quantisation and further edge optimisation. The interaction between quantisation and fixed-point iteration convergence is an open question discussed in Chapter 3. Empirical evaluation of INT8 and FP16 quantisation on both accuracy and Pi 5 inference latency would contribute valuable data.

Scaling the Weighted Maze. Testing on larger grids (20×20, 30×30) with wider weight ranges would reveal how accuracy scales with task complexity. It would also be interesting to test whether a model trained on small grids generalises to larger ones at inference time.

Post-processing for path prediction. The high cell accuracy on Weighted Maze (over 98%) suggests that extracting the lowest-cost connected path from the model's soft output predictions could improve puzzle accuracy without retraining.

Broader edge device evaluation. Extending deployment to other ARM platforms (NVIDIA Jetson Nano, Apple Silicon via CoreML) would provide a wider picture of HRM edge deployment viability.

Adaptive reasoning step count. The trajectory visualisation shows convergence by Step 5 out of 16 for many mazes. Implementing early stopping based on convergence detection could reduce inference latency without sacrificing accuracy, which is particularly relevant for edge deployment where every millisecond counts.

6.7 Final Remarks

This project provides contributions to the emerging field of iterative latent reasoning. The ONNX export pipeline demonstrates that iterative reasoning models can be converted to portable formats despite their dynamic computation patterns, the Weighted Maze task extends HRM evaluation into cost-sensitive domains, and the Raspberry Pi 5 deployment confirms that compact reasoning models can run on resource-constrained hardware without architectural compromise. The results are broadly consistent with the view that computational depth through iterative refinement matters more than hierarchical structure, and as research on alternatives to autoregressive token generation continues to develop, the HRM paradigm offers a direction for efficient, compact, and deployable reasoning systems.

Bibliography

[1] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.

[2] Pierrick Pochelu. Deep learning inference frameworks benchmark. *arXiv preprint arXiv:2210.04323*, 2022.

[3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[4] Rongwu Xu, Zehan Qi, and Wei Xu. Preemptive answer "attacks" on chain-of-thought reasoning. *arXiv preprint arXiv:2405.20902*, 2024.

[5] Xinyun Chen et al. Premise order matters in reasoning with large language models. *arXiv preprint arXiv:2402.08939*, 2024.

[6] William Merrill and Ashish Sabharwal. The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 2023.

[7] Jeffrey Seely, Yuki Imajuku, Tianyu Zhao, Edoardo Cetin, and Llion Jones. Sudoku-bench: Evaluating creative reasoning with sudoku variants. *arXiv preprint arXiv:2505.16135*, 2025.

[8] Renee Ge, Qianli Liao, and Tomaso Poggio. Hierarchical reasoning models: Perspectives and misconceptions. *arXiv preprint arXiv:2510.00355*, 2025.

[9] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. *arXiv preprint arXiv:2505.11831*, 2025.

[10] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[11] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

[12] Jonas Geiping, Tom Goldstein, et al. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.

[13] International Energy Agency. Energy and AI. Technical report, IEA, Paris, 2025.

[14] Zhengyang Geng, Xin-Yu Zhang, Shaojie Bai, Yisen Wang, and Zhouchen Lin. On training implicit models. *arXiv preprint arXiv:2111.05177*, 2021.

[15] Timothy P. Lillicrap and Adam Santoro. Backpropagation through time and the brain. *Current Opinion in Neurobiology*, 55:82–89, 2019.

[16] John D. Murray, Alberto Bernacchia, David J. Freedman, Ranulfo Romo, Jonathan D. Wallis, Xinying Cai, Camillo Padoa-Schioppa, Tatiana Pasternak, Hyojung Seo, Daeyeol Lee, and Xiao-Jing Wang. A hierarchy of intrinsic timescales across primate cortex. *Nature Neuroscience*, 2014.

[17] Timothy P. Lillicrap, Adam Santoro, Luke Marris, Colin J. Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

[18] Mattia Rigotti, Omri Barak, Melissa R. Warden, Xiao-Jing Wang, Nathaniel D. Daw, Earl K. Miller, and Stefano Fusi. The importance of mixed selectivity in complex cognitive tasks. *Nature*, 497:585–590, 2013.

[19] Valerio Mante, David Sussillo, Krishna V. Shenoy, and William T. Newsome. Context-dependent computation by recurrent dynamics in prefrontal cortex. *Nature*, 503:78–84, 2013.

[20] Lorenzo Posani et al. Rarely categorical, always high-dimensional: How the neural code changes along the cortical hierarchy. *bioRxiv*, 2024.

[21] Pablo Villalobos et al. Will we run out of data? limits of LLM scaling based on human-generated data. *arXiv preprint arXiv:2211.04325*, 2022.

[22] Sandeep Kumar, Sanjay Gupta, and Sanjay Arora. Faster scalable ML model deployment using ONNX. In *IEEE CCWC*, 2021.

[23] Yekun Ke, Xiaoyu Li, Yingyu Liang, Zhenmei Shi, and Zhao Song. Advancing the understanding of fixed point iterations in deep neural networks: A detailed analytical study. *arXiv preprint arXiv:2410.11279*, 2024.

[24] Wenxin Jiang et al. Interoperability in deep learning: A user survey and failure analysis of ONNX model converters. In *ACM ISSTA*, 2024.

[25] Hyoukjun Kim et al. Extending the ONNX runtime framework for processing-in-memory execution. In *IEEE ISPASS*, 2022.

[26] Yiming Zhang et al. ONNXPruner: ONNX-based general model pruning adapter. *IEEE Transactions on Neural Networks and Learning Systems*, 2024.

[27] Yang Wang et al. Benchmarking edge AI platforms: NVIDIA Jetson and Raspberry Pi 5. In *IEEE ICICT*, 2024.

[28] Paul-Edouard Novac et al. Quantization and deployment of deep neural networks on microcontrollers. *Sensors*, 21(9):2984, 2021.

[29] Hyungjun Ahn, Jiyong Shin, and Jinwon Park. Performance characterization of using quantization for DNN inference on edge devices. *arXiv preprint arXiv:2303.05016*, 2023.

[30] Xiaoyu Zhao et al. Edge-MPQ: Layer-wise mixed-precision quantization for edge computing. *IEEE Transactions on Computers*, 73(11):2498–2511, 2024.

[31] Zhenzhen Zhang et al. Feasibility analysis of machine learning optimization on GPU-based low-cost edges. In *IEEE CCIS*, 2021.

Appendix A

GitHub Repository

The full source code, documentation, trained models, and screencast demonstration for this project are available at:

`https://github.com/fionntmcc/cross-platform-hrm`

The repository contains the complete Python source code for both model variants (full HRM and L-module-only), dataset generators for Sudoku and Weighted Maze tasks, training and evaluation scripts, the ONNX export pipeline and zero-dependency inference script, CI/CD pipeline configuration (GitHub Actions with Black, Ruff, pytest), unit and integration tests, installation instructions and documentation, and a screencast demonstration of the working software.